算法设计与分析（第四讲）

黄 橙

计算与智能创新学院

chuang@fudan.edu.cn

课程总览

判断一个算法好不好

正确性，复杂度，均摊分析

介绍基本算法思想

分治，随机，数据流算法

学习经典算法范式

动态规划，贪心，回溯

求解网络和路径类问题

最短路径，网络流，二分匹配

求解复杂算法问题

线性规划，在线算法，AI算法

理解高效计算的边界

P vs NP，NP 完全性，近似算法

如何快速找到我要的数据?

学号	姓名
20261341	张三
20262231	李四
20263413	王五
20264313	龙傲天
…	…
20260341	刘波

给你 100 万个学生的信息，现在要查询学号 20261234，怎么办?

抽象成字典

- 我们其实每天都在做这种事情：根据手机号找联系人、根据用户名找用户、根据域名找 IP、根据商品 ID 找商品
- 它们背后其实都是**同一个抽象问题**：从 一个 Key 出发，找一个Value
  - $Search(k)$ 查询
  - $Insert(k, v)$ 插入
  - $Delete(k)$删除
  - $Member(k)$ 是否存在

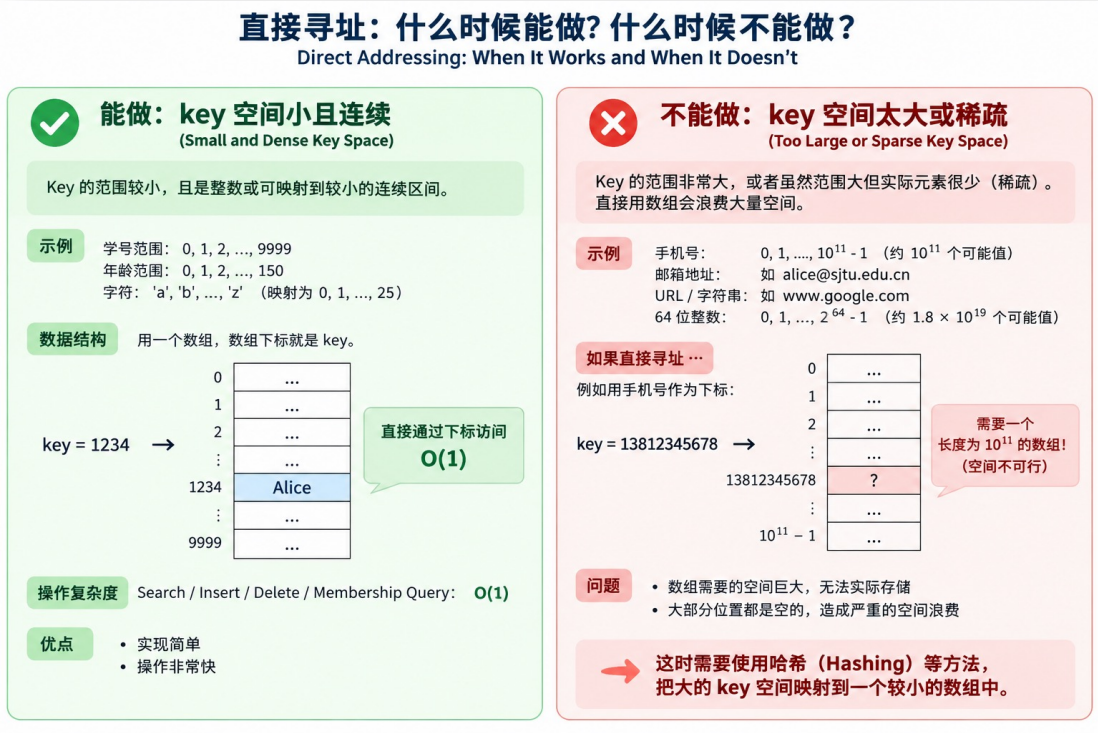
字典怎么构建?

方法	Search	Insert	Delete
未排序的数组	$O(n)$	$O(1)$	$O(n)$
排序的数组	$O(n)$	$O(n)$	$O(n)$
平衡二叉搜索树	$O(\log n)$	$O(\log n)$	$O(\log n)$
?	$O(1)$	$O(1)$	$O(1)$

我们能不能做到，不管有 1 万条还是 100 万条数据， 给我一个 key，我都差不多直接找到它？

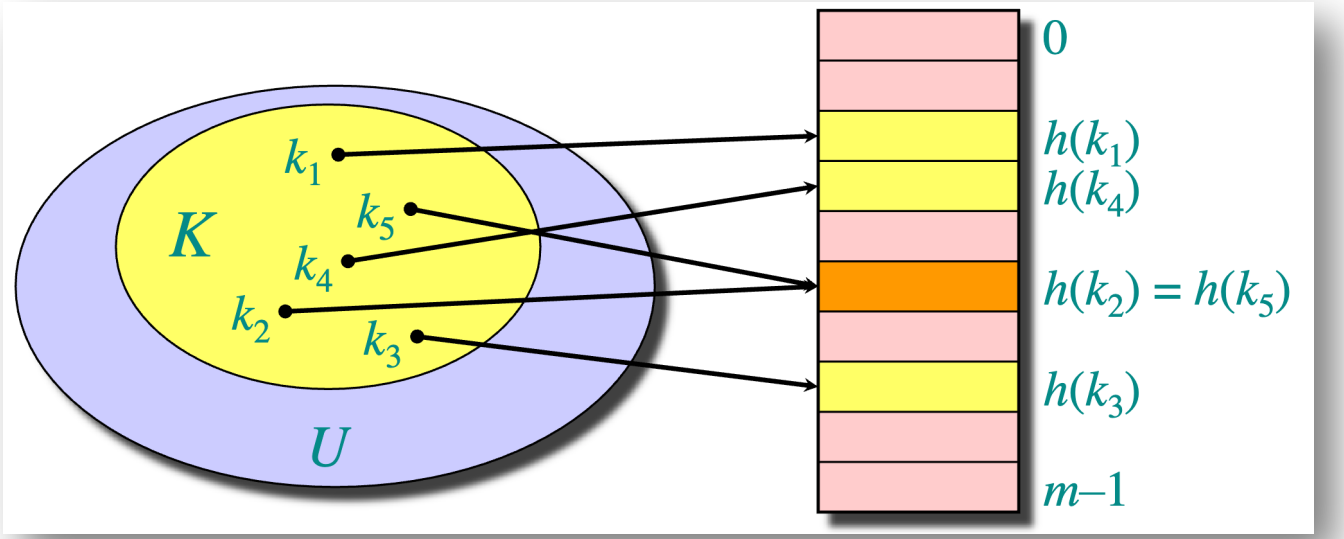
最理想的情况：直接寻址

- 假设学生学号只有： $0,1,2,\ldots,9999$
- 那最简单的方法就是：$table[student_{id}]$
- 比如：$key = 1234$，可以直接输出$table[key]$
- 此时：$Search = Insert = Delete = O(1)$
- **问题：键值范围很大**
  - **空间会浪费**



哈希（Hash）

- 哈希的核心思想：**把巨大的 key 空间，压缩映射到一个较小的数组中。**
- 使用一个哈希函数 h （散列函数）将所有键的全集 U 映射到哈希表 T 中的范围$(1,2,\ldots,m-1)$：



数组 *T*

哈希的本质：将一个大的 键空间 映射 到一个 较小的有限范围

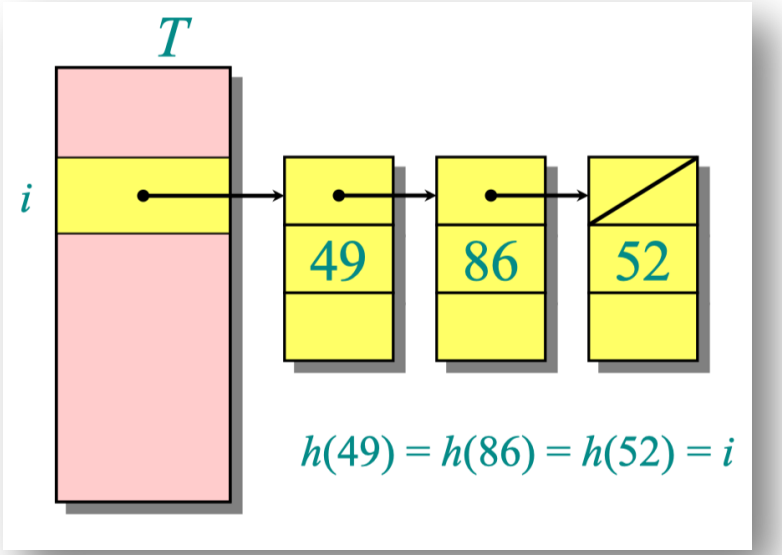
问题：映射可能会产生冲突

如何解决冲突?

- **Separate Chaining（链地址法 / 拉链法）**
- **Open Addressing（开放寻址法）**
  - Linear Probing - 线性探测
  - Quadratic Probing – 二次探测
  - Double Hashing – 双重哈希

链地址法（链表法）：通过链表

- 将映射到同一个位置（Slot）的记录**链接起来**
- **头插法：**在哈希表的链地址法中，新插入的元素通常采用**头插法**（即每次都插入到链表的最前端）



最坏情况: 所有输入（键）都被映射到同一个位置

- 时间复杂度是$O(n)$

平均情况下链地址法的时间复杂度分析

- 假设存在一个简单均匀哈希函数（**本身就很难构造**）：
  - 每一个键 $k \in K$, 被以相同的概率映射到哈希表 T 中的各个位置，与其他键被哈希到的位置无关
  - n 是关键字（键）的个数，m是 数组T 的大小
  - 定义 T 的**负载因子**为每个哈希槽位上的平均键数

$$\alpha = \frac{n}{m} = E(m_j)$$

- 要**分成2种情况讨论**：不成功的查找 和 成功的查找

平均情况下链地址法的时间复杂度分析

- 一次不成功的查找的平均开销是$O(1+\alpha)$，调用哈希函数来访问槽位开销为$O(1)$，搜索链表开销为$O(\alpha)$
- 如果我们期望时间复杂度$=O(1)$, 则只有条件是$\alpha = O(1)$或者$n = O(m)$，才满足
- 成功查找的复杂度分析相对复杂：
  - 设x_i是插入列表中的第i个元素，$i = 1,2,\dots,n$
  - 对键x_i和x_j，定义一个指示器随机变量

$$X_{ij} = \begin{cases} 1 & h(x_i) = h(x_j) \\ 0 & \text{其他} \end{cases}$$

  - 简单均匀哈希：$\Pr\{h(x_i) = h(x_j)\} = \frac{1}{m} \Rightarrow E(X_{ij}) = \frac{1}{m}$
  - **成功查找到第 i 个插入的元素 x_i 时，总共需要检查的元素数目 为** $1 + \sum_{j=i+1}^{n} X_{ij}$

平均情况下链地址法的时间复杂度分析

- 这个公式由两部分组成：1和 $\sum_{j=i+1}^{n} X_{ij}$
- 核心逻辑：**在成功查找时，只有在 x_i *之后* 插入且与 x_i 发生哈希冲突的元素，才会排在 x_i 的前面。**
- 设元素的插入顺序为：$x_1, x_2, \ldots, x_i, \ldots, x_n$，当我们在链表中查找 x_i 时：
  - **在 x_i 之前插入的元素**（如 $x_1, \ldots, x_{i-1}$）：即使它们和 x_i 冲突了，因为它们先来，会排在链表的**更靠后/更深**的位置。我们在从头遍历链表时，会先遇到 x_i，因此这些元素**不会**被检查到。
  - **在 x_i 之后插入的元素**（如 $x_{i+1}, \ldots, x_n$）：如果它们和 x_i 发生了冲突（即 $X_{ij} = 1$），由于头插法，它们会被放在 x_i 的**前面**。我们在查找 x_i 时，必须先跨过它们。
  - 因此，只有在 x_i 之后插入（$j > i$）且发生冲突的元素，才会增加查找 x_i 时的检查次数。
- 当我们顺着链表检查完了排在 x_i 前面的所有冲突元素后，下一步就会比对到 x_i 本身。此时比对成功，查找结束。比对它自己也算作一次检查，所以要加上 1。

成功查找所需的平均（期望）时间复杂度

$$E\left[\sum_{i=1}^{n}\frac{1}{n}\left(1+\sum_{j=i+1}^{n}X_{ij}\right)\right]=\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}E[X_{ij}]\right)=\frac{1}{n}\sum_{i=1}^{n}\left(1+\sum_{j=i+1}^{n}\frac{1}{m}\right)=1+\frac{1}{nm}\sum_{i=1}^{n}(n-i)$$

$$=1+\frac{1}{nm}\left(\sum_{i=1}^{n}n-\sum_{i=1}^{n}i\right)=1+\frac{1}{nm}\left(n^2-\frac{n(n+1)}{2}\right)$$

$$=1+\frac{n-1}{2m}=1+\frac{\alpha}{2}-\frac{\alpha}{2n}\Rightarrow\Theta(1+\alpha)$$

如何构造一个”好的哈希函数“

- 一个“好的哈希函数”应该将关键字尽可能均匀地映射到哈希表的所有槽位中
- 关键字分布中的规律性不应影响这种均匀性
  - 相近的关键字也应该被均匀映射
- 关键字的全域一般假设为自然数集：字符串也可以用自然数集表示（ASCII码）

最常用且经典的方法是**多项式哈希（Polynomial Rolling Hash）**

假设我们有一个字符基数，对于一个字符串 $S = c_0c_1c_2 \dots c_k$：

可以把它看作是一个 B 进制的数，转换公式为：

$$\text{自然数} = c_0 \cdot B^k + c_1 \cdot B^{k-1} + \cdots + c_{k-1} \cdot B + c_k$$

(其中 c_i 代表该位置字符的 ASCII 码)

设基数 B = 128（标准 ASCII 码有 128 个），把字符串 "pt" 转换成自然数：

1. 查 ASCII 码表：'p' = 112，'t' = 116。
2. 带入多项式公式计算：
   $$112\times128^1 + 116\times128^0 = 14336 + 116 = 14452$$
3. 这样，字符串 "pt" 就被完美、唯一地转化为了自然数 14452。

过在实际工程中，128 进制由于偏大，字符串稍长就会导致结果超出计算机整数的最大范围（溢出）。
所以像 Java 的 String.hashCode() 故意把基数 B 选为了一个更小的质数 31

一种简单构造：取余法

- 取余：$h(k) = k \ mod \ m$
- 不要选择有 较小除数 d 的 m。大量与 d 同余的键会对均匀性产生不利影响
- 为什么？

取余法的陷阱：选择不当的 m

- 哈希函数：h(k) = k mod m
  - 如果 m 有较小除数 d，很多键都会集中到少量槽位
  - 结果：槽位分布不均，冲突增多
  - 解决：选 m 为质数

- **例子 ：m = 12（含小因子 2、3、4、6）**
  - 关键字集合：所有偶数 {2,4,6,8,10,12,…}
  - 哈希值：k mod 12 ∈ {0,2,4,6,8,10}
  - **只有 6 个槽位在用，另一半永远空置**

k (偶数)	h(k) = k mod 12
2	2
4	4
6	6
8	8
10	10
12	0

- 如$m = 2^r$，那么哈希甚至不会依赖于键 k 的所有位：
  - 例如$k = 101010110011001$, 当$r = 7$, 则$h(k) = 011001$
  - 原因：在二进制中，乘以 2 或除以 2 相当于移动小数点。同理，**对 2^r 取模，本质上就是直接切掉高位，只保留最低的r位。**

取余法

- 选择 m 为一个素数，且该素数不应过于接近 2 的幂次

- 例如：用取余法构造一哈写表，存储2000个元素，并用链表解决冲突，$m = 701$

- **装载因子 (Load Factor)**
  - $\alpha = n/m = 2000/701 \approx 2.85$
  - 平均每个槽位链表长度 ≈ 2.85
  - **成功查找代价** $\approx 1 + \alpha/2 \approx 2.4$**次比较**
  - **不成功查找代价** $\approx 1 + \alpha \approx 3.85$**次比较**

方法2：乘法散列法

- 乘法散列法：$h(k) = \lfloor m(kA \mod 1) \rfloor$（$k$ 假设为 w 位）
- $kA \mod 1$ ⇒取小数部分，$0 < A < 1 \Rightarrow A = \frac{S}{2^w}$
- 好处：更快，易实现，m 的选择再不重要
  - 设$m = 2^p$

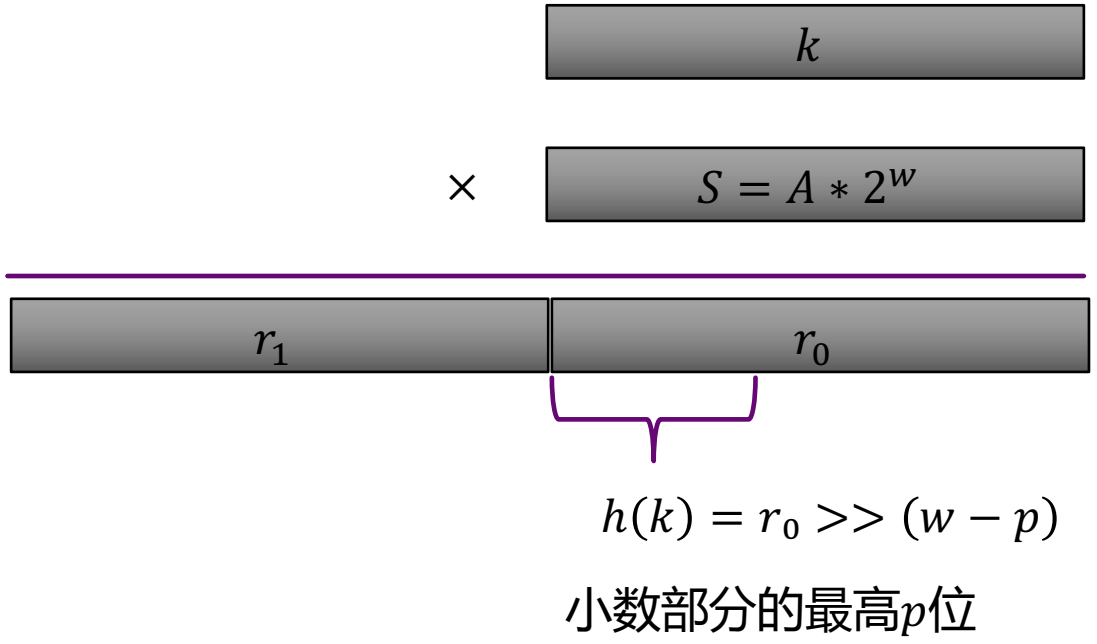
- 关键：序列 $kA \mod 1$ 在 [0,1) 上近似均匀分布
- 小知识：这个公式里最关键的就是常数 A。科学家高德纳（Knuth）经过数学分析发现，当 A**取黄金分割数**时，这个算法的散列效果相对好：$A = \frac{\sqrt{5}-1}{2} \approx 0.6180339887 \ldots$

例子

- 其实就是把计算分成了 4 个小动作：
  - kA：选择一个介于 0 到 1 之间的常数小数 A（比如 $A = 0.618$），用你的关键字 k 乘以它。
  - $(kA \mod 1)$：这步的意思是**“只取乘积的小数部分”**。
    - *举个例子*：如果 $k \times A = 13.65$，那么对 1 取模后，整数 13 被丢弃，只留下**小数部分 0.65**。这个结果永远在 $[0,1)$之间。
  - m×乘积的小数部分：把这个 $[0,1)$之间的小数，乘以哈希表的大小 m。这相当于把数值等比例放大到了 $[0, m)$的区间内。
    - *接着上面的例子*：如果哈希表大小 $m = 100$，那么 $100 \times 0.65 = 65.0$。
  - $\lfloor \ldots \rfloor$（**向下取整**）：切掉最后算出来数字的小数位，只留整数。
    - *最后结果*：65.0取整后就是 **65**。这就是最终的哈希槽位。

速度快的原因

- 把A看成一个 **定点数（fixed-point number）**
  - 令 $S = \lfloor A \cdot 2^w \rfloor$：$A \approx \frac{s}{2^w}$
- kA 可以写成：$kA \approx \frac{ks}{2^w}$
- 两个$w-bit$整数相乘：$k \times s$ 结果最多是 $2w\ bit$
- 因此可以拆成：$ks = r_1 \cdot 2^w + r_0$
  - 如果：$m = 2^p$
  - 那么：$h(k) = \left\lfloor 2^p \frac{r_0}{2^w} \right\rfloor$
  - 即：$h(k) = \left\lfloor \frac{r_0}{2^{w-p}} \right\rfloor$
  - 在计算机里就是：$\boxed{h(k) = r_0 >> (w-p)}$



一些现代哈希算法

- MurmurHash
  - 是一个高性能、分布均匀的哈希函数，适用于大数据量的哈希表，被广泛应用于分布式系统和数据库系统
- CityHash
  - 由 Google 开发的哈希函数，设计用于处理不同长度的字符串，对短字符串和长字符串都进行了优化处理
- XXHash
  - 是一种非常快速的非加密哈希算法，主要用于需要高性能的场景，比如文件校验、数据压缩、缓存等

什么是 **MurmurHash**?

- 非加密哈希函数，由 Austin Appleby 在 2008 年提出
- 名称由 “murmur” (模糊) + “mash” (捣碎) 演变而来
- 目标：**高性能、分布均匀、低碰撞率**
- 广泛应用于：
  - 大规模哈希表（分布式系统、数据库）
  - 数据流处理（如 Hadoop, Cassandra, Redis 内部用过）
  - 非安全场景的 哈希（不同于 MD5、SHA）

从MurmurHash看：一个现代哈希，是怎么把有规律的输入变成近似随机的输出的?

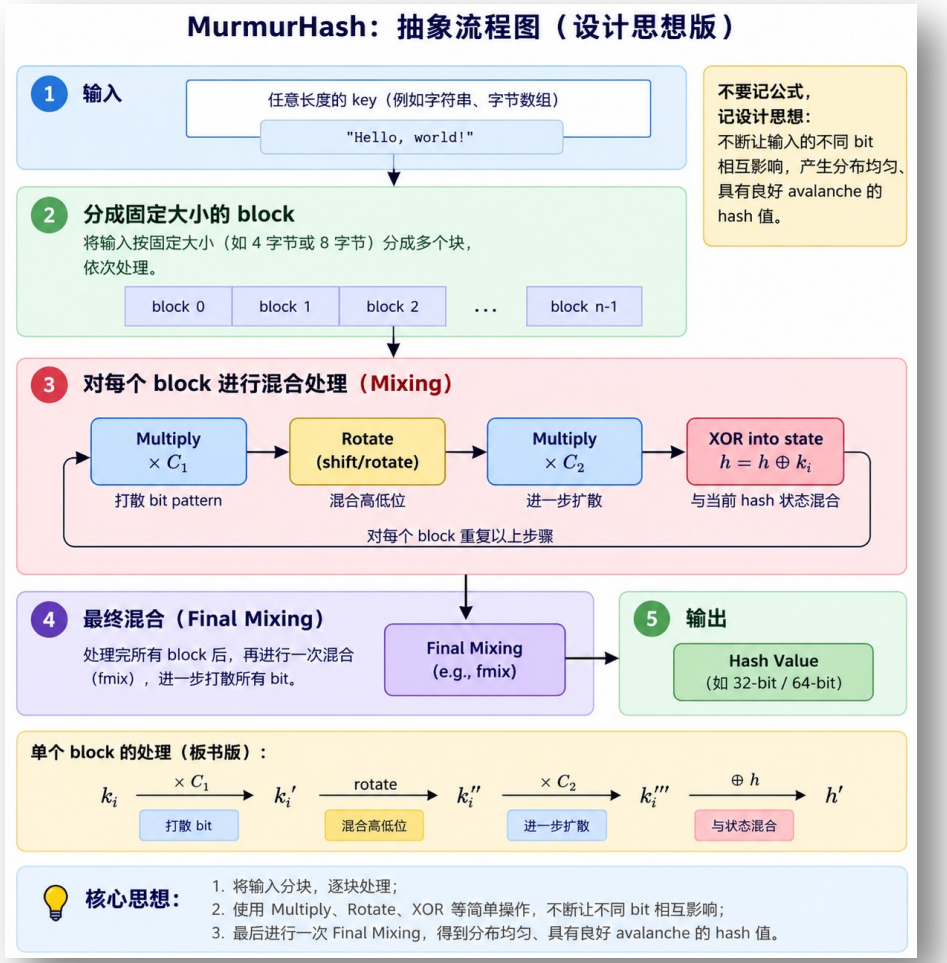
- MurmurHash 基本操作：
  - 乘法 (× 常数) → 打散位模式
  - 移位 (>>，<<) → 混合高低位
  - 异或 (⊕) → 消除相关性
- 思路：让输入的每一位都对输出产生“雪崩效应”
- 雪崩效应（Avalanche Effect）：输入任意 1 bit 改变 → 输出大约 50% 的 bit 改变

假设：$x = 00101101$
计算：$x \times C$
常数 C的二进制是： $C = 1011_2$
乘法其实相当于：$x \times C = x + (x \ll 1) + (x \ll 3)$

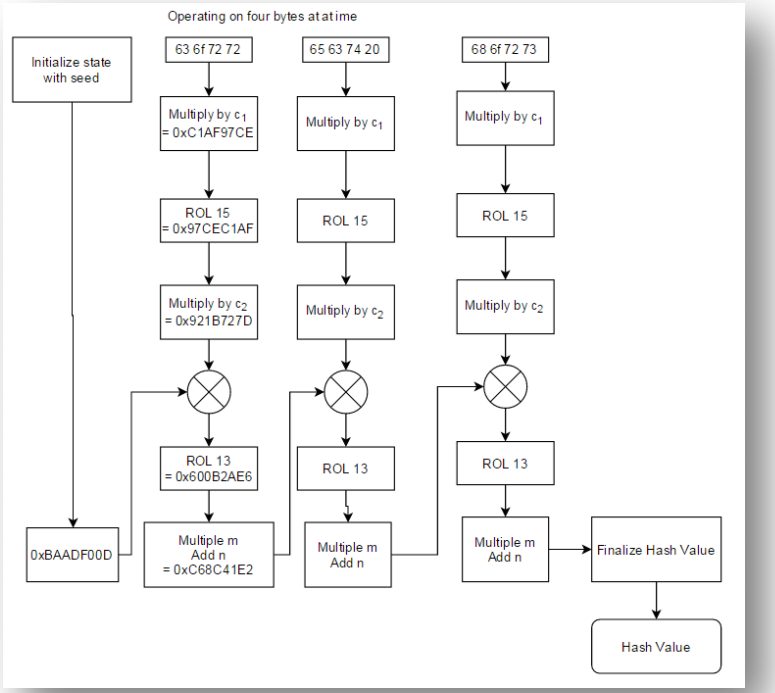
```
x       00101101
x<<1    01011010
x<<3   1 01101000
-------------------
       加在一起
```

操作	作用
Multiply	通过移位副本叠加和进位，让一个 bit 影响多个位置
Rotate / Shift	重新排列 bit，使高低位进入不同位置参与下一轮 mixing
XOR	将不同来源的信息合并到当前 hash state

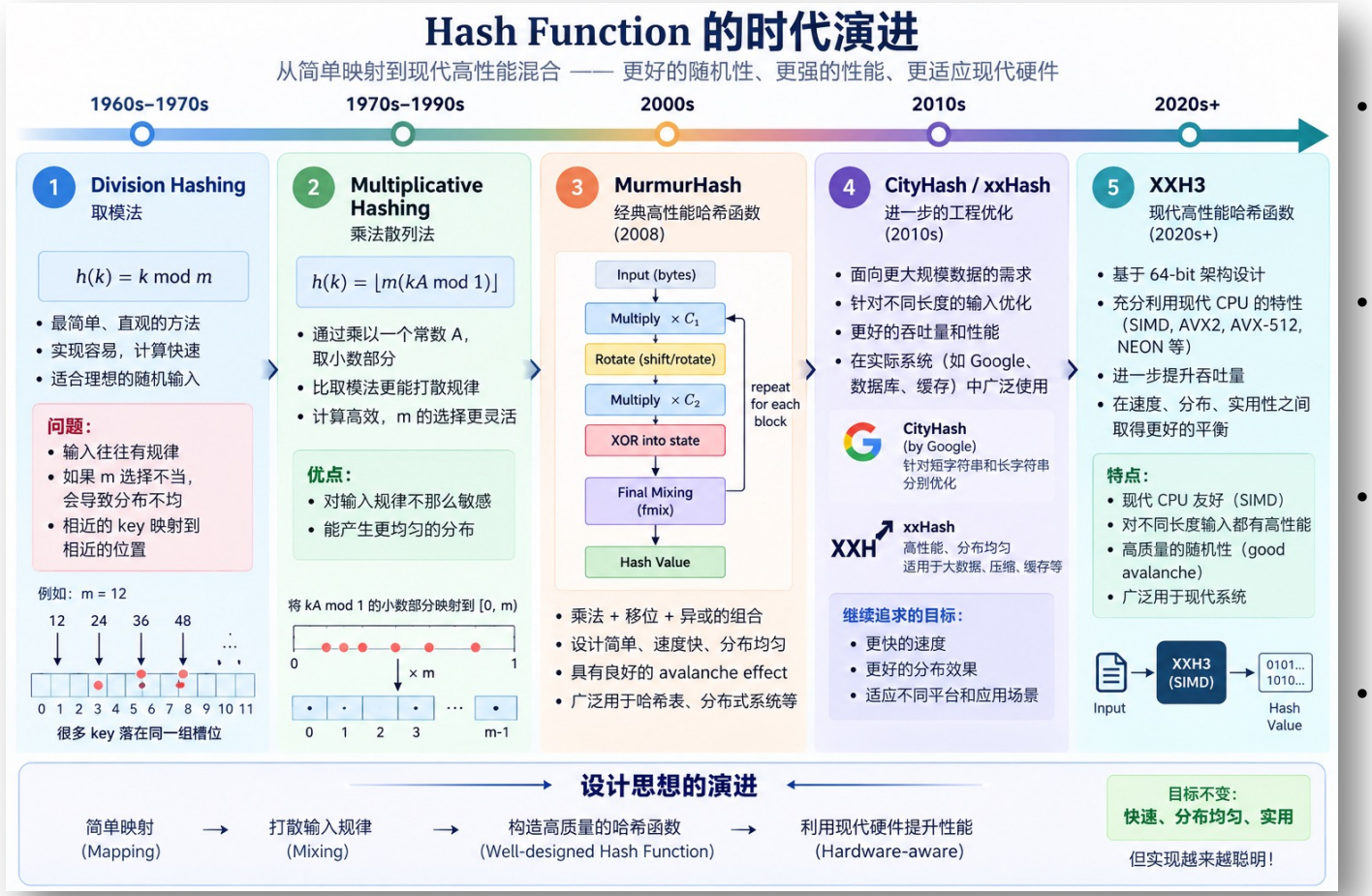
MurmurHash算法的设计思想



1. 输入分块（每次处理 4 字节 -- 32 位版本）
2. 每块执行：
    1. k × C1（常数）
    2. 左移 15 位
    3. × C2（常数）
    4. 与哈希值混合 + seed
    5. 左移 13 位
    6. h = h * m + n （进一步打散：m=5, n=0xe6546b64）
3. 所有块处理完 → 处理剩余字节
4. 最终混合（fmix）：
    1. h ^= h >> 16
    2. h *= 0x85ebca6b
    3. h ^= h >> 13
    4. h *= 0xc2b2ae35
    5. h ^= h >> 16
5. 输出 32 位 hash 值



哈希算法的时代演进



- **Length-aware（长度感知）**
  针对短、中、长输入采用不同处理路径，而不是所有数据都走同一套流程。
- **Wide Mixing（宽位混合）**
  大量使用 64-bit/128-bit arithmetic，让更多输入 bit 在一次计算中相互影响，提升 diffusion 和 avalanche。
- **Parallel Accumulators（多路并行累加）**
  长输入使用多个 accumulator 同时处理不同 data lane，减少串行 dependency。
- **SIMD-friendly / Hardware-aware（面向现代 CPU）**
  计算结构适合 AVX2、AVX-512、NEON 等 SIMD 指令，提高吞吐量。

开放寻址法

- **区别于链地址发法**
- 不使用哈希表之外的存储空间
  - 插入时系统地探查哈希表，直到找到一个空槽
  - **开放寻址法里，同一个 key 不只对应一个槽位，而是对应一整条“探查序列”**。所以哈希函数除了看 key，还要看“这是第几次尝试”，也就是探查次数 i。：

$$h(\mathrm{k},\mathrm{i}): U\times\{0,1,\dots,m-1\}\rightarrow\{0,1,\dots,m-1\}$$

- 探查序列 $<h(k,0),h(k,1),\dots,h(k,m-1)>$ 应该是序列 $\{0,1,\dots,m-1\}$ 一种排列
- 哈希表可能会被填满，删除操作较为困难（因为是探查找到的）

探查策略：线性探查

- 线性探查（Linear Probing）
- 给定一个一般的哈希函数 $h'(k)$，线性探查使用的哈希函数为：
- $h(k,i) = (h'(k) + i)\ mod\ m, i = 0,1,2 \ldots, m-1$

- 这种方法虽然简单，但存在**主聚类**问题，即连续占用的槽位会逐渐增多，从而增加平均查找时间。此外，已占用槽位的长串往往会变得更长。

例子

开放寻址法：Linear Probing

哈希表大小 m = 10 · h(k) = k mod 10



依次插入：586 → 133 → 204 → 481 → 496

思想：冲突时不使用链表，而是在表内按探查序列寻找下一个空槽。

主聚类问题

- 当很多元素连续地堆积在相邻槽位上，形成一段“长块”。
- 新插入的元素如果哈希到这段区间附近，就必须探测整个长块才能找到空槽。
- 结果：这个长块会越变越长，查找和插入代价变大。
- 例子：哈希表大小 m=10，使用线性探测：
  - 插入 key1 → 槽 3
  - 插入 key2 → 也映射到槽 3 → 冲突 → 放到槽 4
  - 插入 key3 → 槽 4 (冲突) → 放到槽 5
  - 插入 key4 → 槽 5 (冲突) → 放到槽 6
- 结果槽 3–6 形成一个连续的“聚集区”。
- 再插入任何哈希到 3–6 的 key，都要顺着走完一段长链，效率急剧下降。

主聚类


1
插入 13
h(13) = 13 mod 10 = 3
槽 3 为空，直接插入。
0 1 2 3 4 5 6 7 8 9
13
2
插入 23
h(23) = 23 mod 10 = 3
槽 3 已被占用，线性探查到槽 4，插入。
0 1 2 3 4 5 6 7 8 9
13 23
3
插入 33
h(33) = 33 mod 10 = 3
槽 3、4 已被占用，探查到槽 5，插入。
0 1 2 3 4 5 6 7 8 9
13 23 33
4
插入 43
h(43) = 43 mod 10 = 3
槽 3、4、5 已被占用，探查到槽 6，插入。
0 1 2 3 4 5 6 7 8 9
13 23 33 43
5
插入 53
h(53) = 53 mod 10 = 3
槽 3、4、5、6 已被占用，探查到槽 7，插入。
0 1 2 3 4 5 6 7 8 9
13 23 33 43 53
6
形成主聚类
0 1 2 3 4 5 6 7 8 9
13 23 33 43 53
主聚类（连续的已占用槽位）
主聚类现象
不仅那些哈希到 3 的键会落入这个连续区域。
例如：如果插入 24（h(24) = 4）、25（h(25) = 5）等，同样会被探查到这个区域并插入到后面。
因此，这个连续的 cluster 会越来越长，这种现象称为 主聚类（Primary Clustering）。
直观理解：
一旦形成一段连续的已占用槽位，
它就会像“磁铁”一样吸引更多的键，
使得这段连续区域不断扩大。
主聚类会导致探查序列变长，查找、插入的性能下降。

探查策略：二次探查

- 二次探查
- 给定一个一般的哈希函数 $h'(k)$，二次探查使用的哈希函数为：
- $h(k,i) = (h'(k) + c_1 i + c_2 i^2) \, mod \, m, i = 0,1,2 \ldots, m-1$

- 比线性探查好，但也有问题
  - 初始探查位置相同，后续也相同，存在**二次聚类**问题
  - 如果两个不同的 key 初始哈希值相同，它们的探测序列完全相同（$1^2, 2^2, 3^2 \ldots$）。这意味着它们必然冲突在同一条探测路径上 → 会聚在一起。这种现象叫**二次聚类**。

二次聚类的例子

例子： 设哈希表大小 $m = 11$，$h'(k) = k \bmod 11$，采用二次探查 $h(k,i) = (h'(k) + i^2) \bmod 11$ （即 $c_1 = 0, c_2 = 1$）

插入键 22，33，44，55。它们的初始哈希位置均为 0（因为 $22 \bmod 11 = 33 \bmod 11 = 44 \bmod 11 = 55 \bmod 11 = 0$）。

1 插入 22

探查序列：0, 1, 4, 9, 3, 3, ...

$i = 0$:

$h(22,0) = (0 + 0^2) \bmod 11 = 0$

槽 0 为空，插入 22。

0	1	2	3	4	5	6	7	8	9	10
22										

2 插入 33

探查序列：0, 1, 4, 9, 5, 3, ...

$i = 0$: $h(33,0) = 0$ （已被占用）

$i = 1$: $h(33,1) = (0 + 1^2) \bmod 11 = 1$

槽 1 为空，插入 33。

0	1	2	3	4	5	6	7	8	9	10
22	33									

3 插入 44

探查序列：0, 1, 4, 9, 5, 3, ...

$i = 0$: $h(44,0) = 0$ （已被占用）

$i = 1$: $h(44,1) = 1$ （已被占用）

$i = 2$: $h(44,2) = (0 + 2^2) \bmod 11 = 4$

槽 4 为空，插入 44。

0	1	2	3	4	5	7	8	9	10
22	33			44					

4 插入 55

探查序列：0, 1, 4, 9, 5, 3, ...

$i = 0$: $h(55,0) = 0$ （已被占用）

$i = 1$: $h(55,1) = 1$ （已被占用）

$i = 2$: $h(55,2) = 4$ （已被占用）

$i = 3$: $h(55,3) = (0 + 3^2) \bmod 11 = 9$

槽 9 为空，插入 55。

0	1	2	3	4	5	7	8	9	10
22	33			44				55	

观察与结论

- 22、33、44、55 的初始哈希位置相同（都是 0）。
- 在二次探查下，它们的探查序列完全相同：0, 1, 4, 9, 5, 3, ...
- 这些键不断在同一组槽位上发生冲突，形成**二次聚类**（Secondary Clustering）。
- 二次探查避免了线性探查的“主聚类”（不同起点的键也会挤成连续块），但仍然存在**二次聚类**：

  —— 具有相同初始位置的键会沿着相同的二次探查序列竞争同一组槽位。

如何规避聚类?

聚类的本质：探查序列高度相关。

- **Linear Probing**：固定步长为 1，不同 key 的探查路径容易汇合
  → **Primary Clustering**
- **Quadratic Probing**：相同初始哈希位置会产生相同探查序列
  → **Secondary Clustering**

一个直观想法：**不仅让 key 决定起点，也让 key 决定探查步长**

探查策略：双重哈希

- 双重哈希

- 给定两个一般的哈希函数 $h_1(k), h_2(k)$，双重探查使用的哈希函数为：
- $h(k,i) = (h_1(k) + ih_2(k)) \ mod \ m, i = 0,1,2 \ldots, m-1$

- 这种方法通常能产生很好的结果，但$h_2(k)$最好与m互质
- **为什么？**

不互质的情况分析

- $h_2(k)$**决定探查步长**
- 如果 $h_2(k)$ 与 m 有公因数 $d > 1$：
  - 探查序列只能落在 m/d 个槽位
  - 不能覆盖整个表
  - 可能出现“找不到空位”的死循环
- **若** $h_2(k)$ **与** m **互质**：
  - 保证序列能遍历表中所有 m 个槽位
  - 冲突解决一定成功

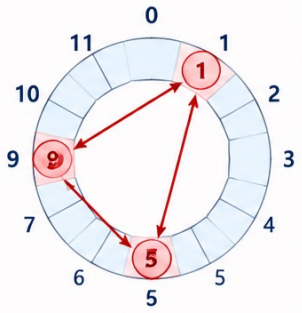
双重哈希（Double Hashing）：步长与表大小互质的重要性

探查序列： $h(k,i) = (h_1(k) + ih_2(k)) \bmod m$ ， $i = 0, 1, 2, \ldots$

要点： 若 $\gcd(h_2(k), m) = 1$，则探查序列可以遍历所有 m 个槽位。

例 1：$h_2(k)$ 与 m 不互质（不能遍历整个表）

$m = 12$
$h_1(k) = 5$
$h_2(k) = 4$ (gcd(4, 12) = 4 ≠ 1)



探查序列：

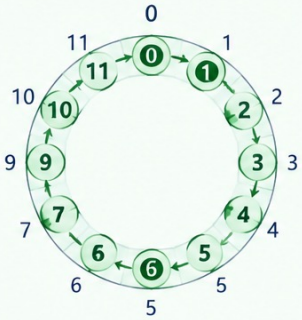
$i = 0$: $(5 + 0 \times 4) \bmod 12 = 5$
$i = 1$: $(5 + 1 \times 4) \bmod 12 = 9$
$i = 2$: $(5 + 2 \times 4) \bmod 12 = 1$
$i = 3$: $(5 + 3 \times 4) \bmod 12 = 5$
$i = 4$: $(5 + 4 \times 4) \bmod 12 = 9$
$i = 5$: $(5 + 5 \times 4) \bmod 12 = 1$
...

序列：5, 9, 1, 5, 9, 1, ...

结果： 只覆盖了 **3 个槽位（5, 9, 1）**，无法访问其他槽位。
如这3个槽位都被占用，即使表中还有空位，也会插入失败。

例 2：$h_2(k)$ 与 m 互质（可以遍历整个表）

$m = 12$
$h_1(k) = 5$
$h_2(k) = 5$ (gcd(5, 12) = 1)



探查序列：

$i = 0$: $(5 + 0 \times 5) \bmod 12 = 5$
$i = 1$: $(5 + 1 \times 5) \bmod 12 = 10$
$i = 2$: $(5 + 2 \times 5) \bmod 12 = 3$
$i = 3$: $(5 + 3 \times 5) \bmod 12 = 8$
$i = 4$: $(5 + 4 \times 5) \bmod 12 = 1$
$i = 5$: $(5 + 5 \times 5) \bmod 12 = 6$
$i = 6$: $(5 + 6 \times 5) \bmod 12 = 11$
$i = 7$: $(5 + 7 \times 5) \bmod 12 = 4$
$i = 8$: $(5 + 8 \times 5) \bmod 12 = 9$
$i = 9$: $(5 + 9 \times 5) \bmod 12 = 2$
$i = 10$: $(5 + 10 \times 5) \bmod 12 = 7$
$i = 11$: $(5 + 11 \times 5) \bmod 12 = 0$

序列：5, 10, 3, 8, 1, 6, 11, 4, 9, 2, 7, 0

结果： 覆盖了所有 12 个槽位，保证在表未满时一定能找到空位。

结论

- 在双重哈希中，必须选择 $h_2(k)$ 使得 $\gcd(h_2(k), m) = 1$，探查序列才能遍历整个哈希表。
- 否则，探查序列只能访问表的一个子集，可能在表未满时就无法插入。

开放寻址法的时间复杂度分析

- 假设**均匀哈希**：每个键值都有相同的可能性被分配到 $m!$ 种探查序列中的任何一种
- 在双重探查里，每个 key 会生成一条 **探查序列**：

$$h(k,0), h(k,1), \ldots, h(k,m-1)$$

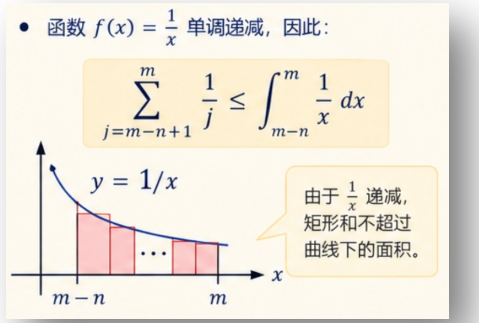
- 总共有 **m! 种可能的探查序列**（因为 m 个槽位的排列组合有 $m!$ 种）。
  - 假设哈希表有 4 个槽：$\{0,1,2,3\}$
  - 一次完整的探查序列就是把 4 个槽位按照某种顺序访问。例如：
    - $0 \to 1 \to 2 \to 3$
    - $0 \to 1 \to 3 \to 2$
    - $0 \to 2 \to 1 \to 3$
    - $0 \to 2 \to 3 \to 1$
    - ...
    - $3 \to 2 \to 1 \to 0$
  - 因为 4 个槽位可以形成：4!=24 种不同排列，所以 均匀哈希 假设：
    - 对任意一个 key，它的完整探查序列等概率地是这 24 种排列中的任意一种。

开放寻址法的时间复杂度分析

- **一次不成功的查找**
- **定理：**给定一个开放寻址的哈希表，负载因子为 $\alpha = \frac{n}{m} < 1$，在一次不成功的查找中，期望的探查次数最多为$\frac{1}{1-\alpha}$
- 至少需要一次探查(n是已存元素个数，m = 哈希表的槽位总数)
  - 以概率$\frac{n}{m}$，第一次探查会碰到一个已占用的槽位，因此需要第2次探查。
  - 以概率 $\frac{n-1}{m-1}$，第二次探查会碰到一个已占用的槽位，因此需要第3次探查。
  - 以概率 $\frac{n-2}{m-2}$，第三次探查会碰到一个已占用的槽位，依此类推。
- 定义随机变量X为一次不成功查找需要的探查次数
- 定义$A_i(i = 1,2,\dots)$事件为第i次探查且探查到的是一个被占用的槽位的
- $\Pr\{A_1\} = \frac{n}{m}, \Pr\{A_i|A_1 \cap A_2 \cap \cdots \cap A_{i-1}\} = \frac{n-i+1}{m-i+1}$
- $\Pr\{X \geq i\} = \Pr\{A_1\} * \Pr\{A_2|A_1\} * \cdots * \Pr\{A_{i-1}|A_1 \cap A_2 \cap \cdots \cap A_{i-2}\}$

开放寻址法的时间复杂度分析

- $\Pr\{X \geq i\} = \frac{n}{m} * \frac{n-1}{m-1} * \frac{n-2}{m-2} * \cdots * \frac{n-i+2}{m-i+2}$，考虑到 $\frac{n-i}{m-i} \leq \frac{n}{m} = \alpha, i = 1,2, \ldots, n$
- 得到：$\Pr\{X \geq i\} \leq \left(\frac{n}{m}\right)^{i-1} = \alpha^{i-1}$
- 因此，得到 $E[X] = \sum_{i=1}^{\infty} i \Pr(X = i) = \sum_{i=1}^{\infty} \Pr\{X \geq i\} \leq \sum_{i=1}^{\infty} \alpha^{i-1} = \frac{1}{1-\alpha}$
- 一次成功的查找
- **定理：**给定一个开放寻址哈希表，负载因子为 $\alpha = \frac{n}{m} < 1$，在一次成功的查找中，期望的探查次数最多为 $\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
- **证明：**k是 第$i+1$个 被插入表中的键，则对k进行查找，探查期望次数最多为 $\frac{1}{1-i/m} = \frac{m}{m-i}$（用到之前定理）
  - 期望 $\frac{1}{n}\sum_{i=0}^{n-1} \frac{m}{m-i} = \frac{1}{\alpha} \ln \frac{m}{m-n} = \frac{1}{\alpha} \ln \frac{1}{1-\alpha}$（积分近似）

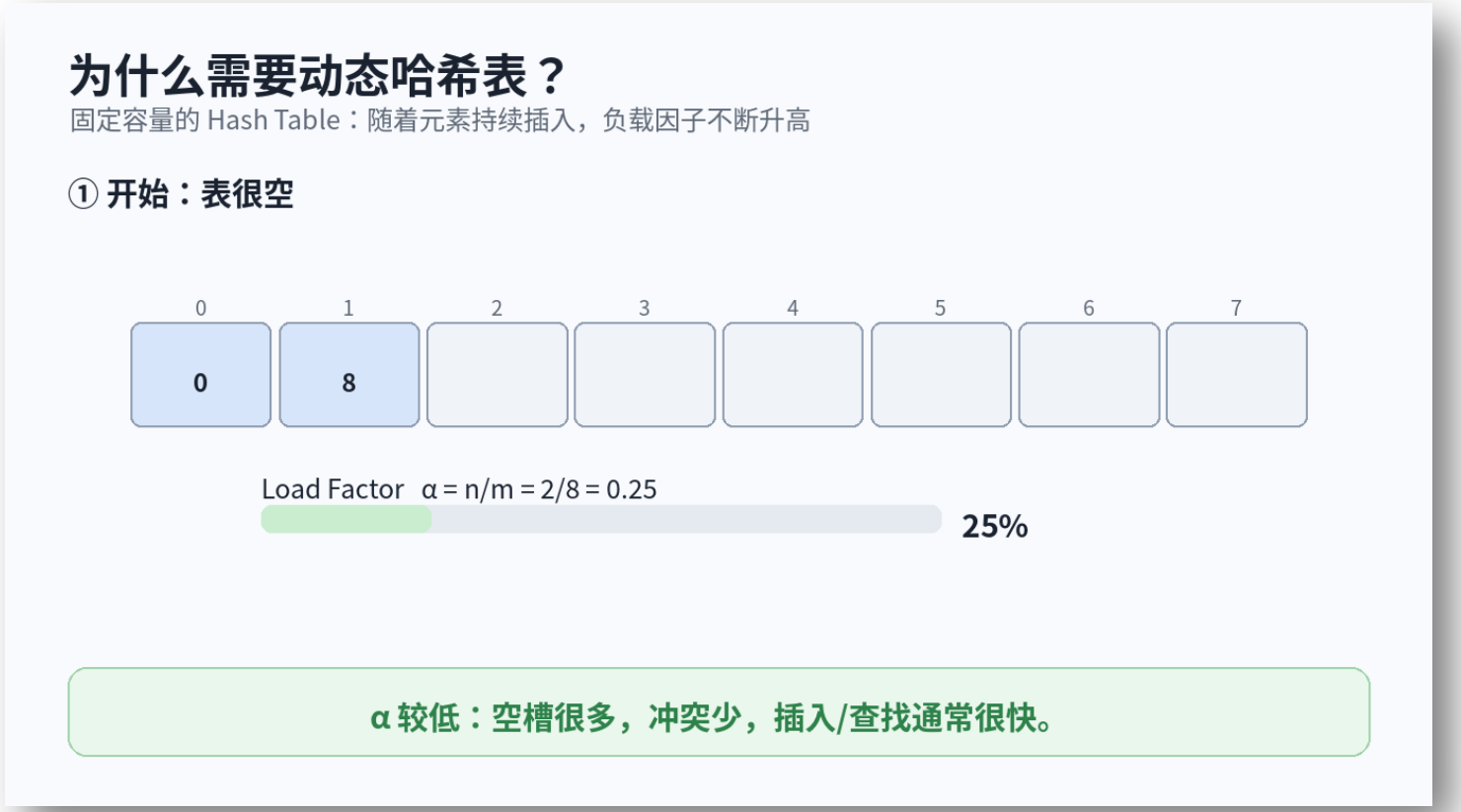


定理的背后

- 如果 α 是常量，则在使用**理想均匀哈希**访问一个开放地址的哈希表的时间复杂度是常数时间。
  - 如果哈希表的负载因子为 0.5（即表满了一半），那么期望的探查次数是 1/(1−0.5)=2。
  - 如果哈希表的负载因子为 0.9（即表满了 90%），那么期望的探查次数是 1/(1−0.9)=10。

方法	平均查找	高负载时	原因
Linear	$O(1)$	恶化最快	Primary Clustering
Quadratic	$O(1)$	较好	Secondary Clustering
Double Hashing	$O(1)$	最接近理想模型	Probe sequence 更独立
Uniform Hashing	$O(1)$	理论基准	理想随机探查

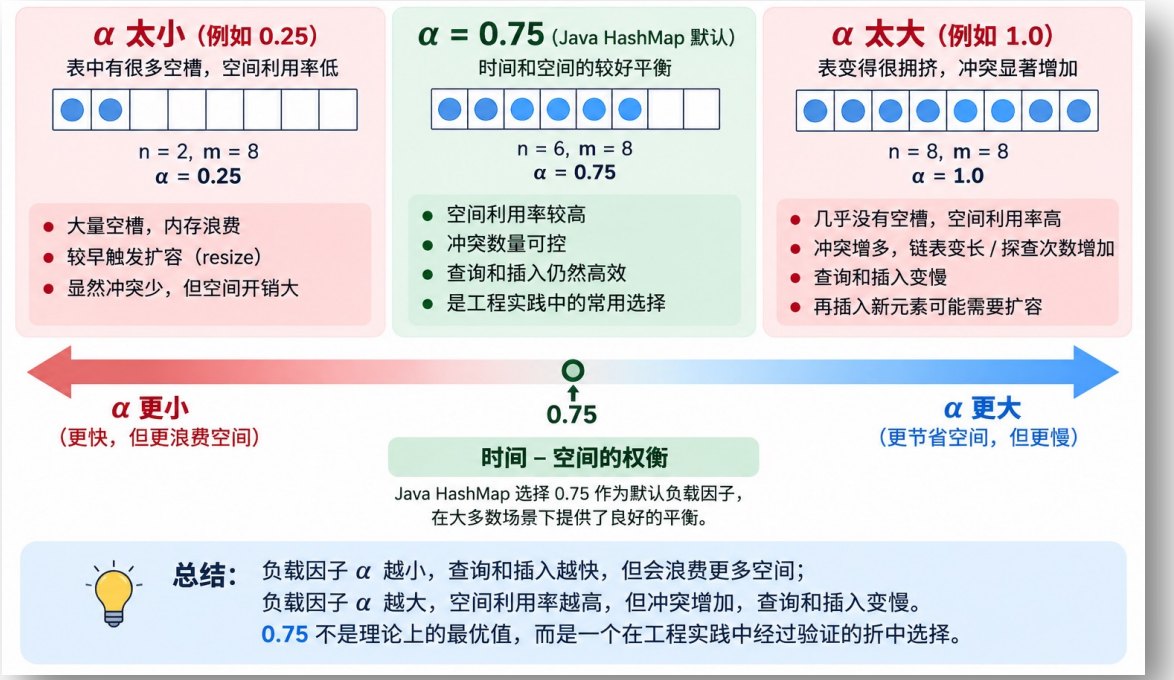
当一个哈希表快满的时候?



随着 n增长，如果 m不变，会发生什么？

什么时候扩容？扩多少？

- 定义：$\boxed{\text{threshold} = m \times \text{load factor}}$，当：$n > \text{threshold}$ 时候，就：$\boxed{m \to 2m}$
- Java HashMap ：Java 目前默认：$\text{initial capacity} = 16$，$\text{load factor} = 0.75$
- 当 entry 数超过 capacity × loadFactor 时，会重建内部结构，容量大约翻倍
- 例如：$m = 16$，$threshold$：$16 \times 0.75 = 12$



扩容后的Rehash

capacity 改变

↓

bucket index 改变

↓

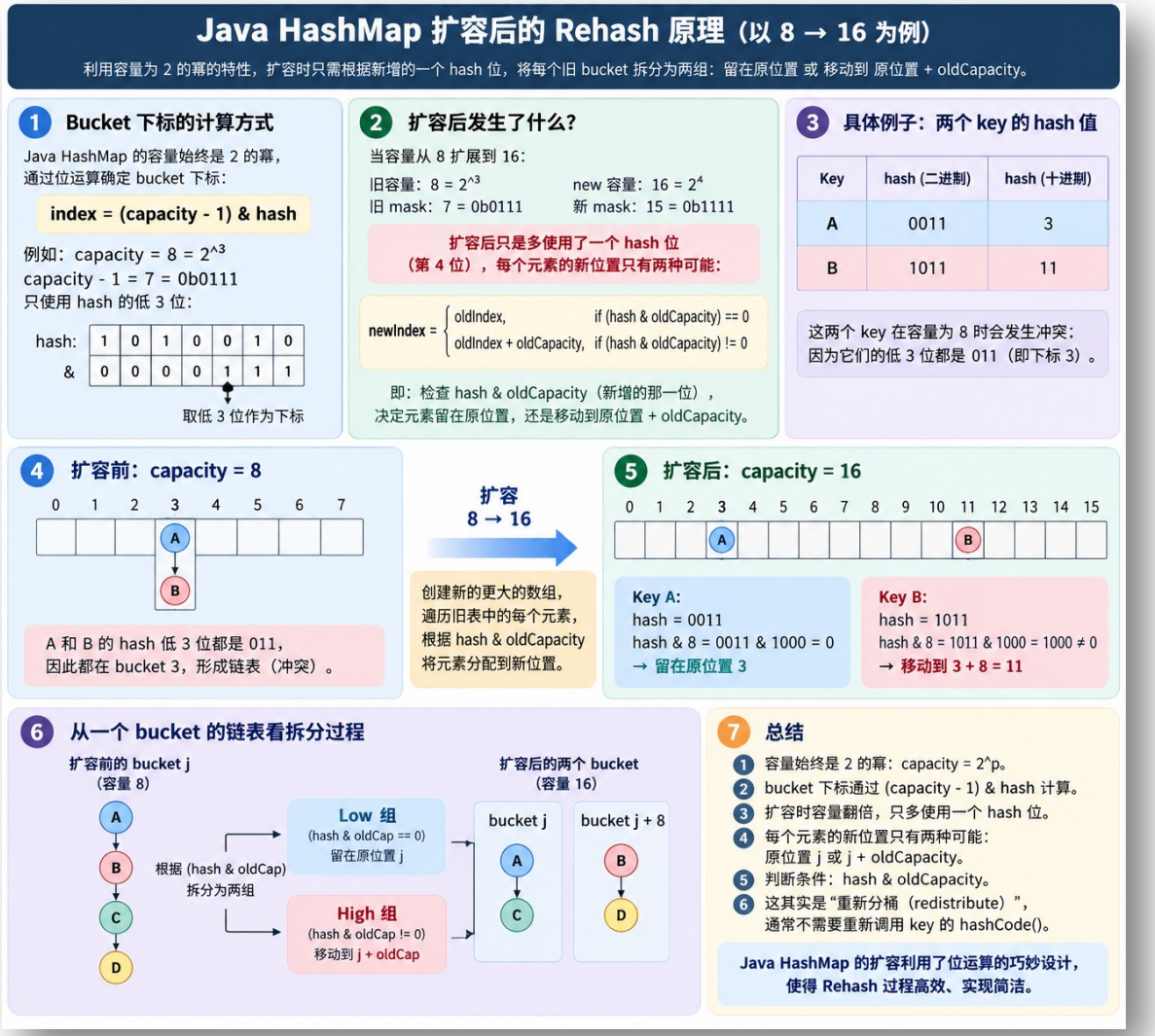
重新分配所有元素

↓

Rehash / Redistribute

Java HashMap的操作

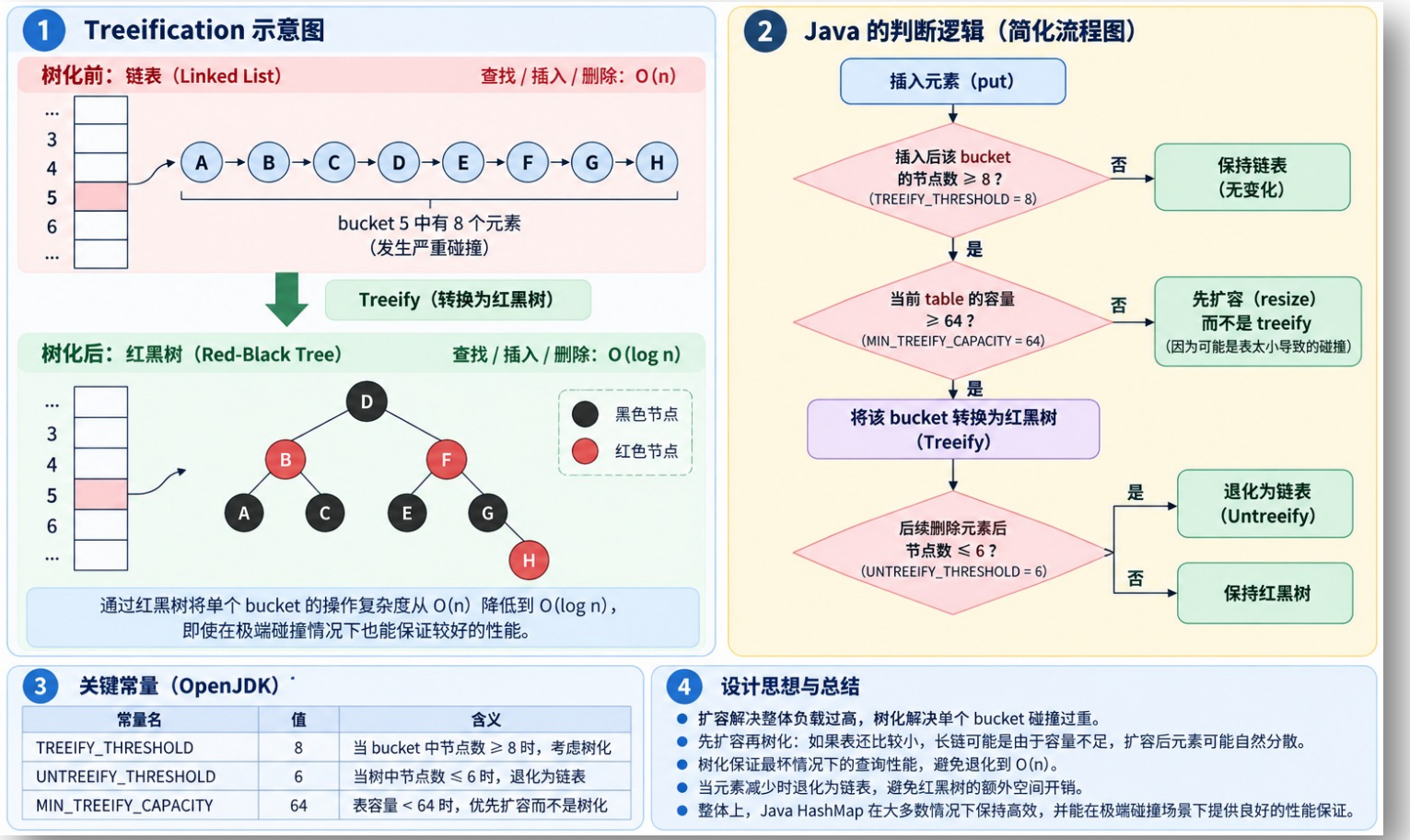
```
h = key.hashCode();          // 先得到一个 32-bit int
hash = h ^ (h >>> 16);       // HashMap 再做轻量 mixing
index = hash & (n - 1);      // 最后定位 bucket
```



Resize 很贵，为什么 Insert 还是 O(1)?

- 连续 n 次插入：$\underbrace{n}_{\text{正常插入}} + \underbrace{\left(1+2+4+\cdots+\frac{n}{2}\right)}_{\text{Resize 搬迁}}$，而：$1+2+4+\cdots+\frac{n}{2} < n$
- 因此：$T(n) < 2n$所以：$\boxed{\frac{T(n)}{n} = O(1)}$
- 记账法类似：
  - 1 元 → 支付当前插入
  - 2 元 → 存起来，为未来 Resize 买单

碰撞太严重怎么办?



问题	怎么处理
整个表太满	Resize
某个 bucket 太长	Treeify
表小且 bucket 太长	优先 Resize
树变得很小	Untreeify

- TreeNode 的空间成本大约是普通 Node 的两倍，所以小 bucket 没必要维护复杂树结构
- 因此如果 resize / remove 以后节点变少，会在适当情况下重新变回普通 bin。

扩容解决“整体拥挤”，树化解决“局部极端碰撞”

Java HashMap 和 HashTable

	HashMap	Hashtable
基本思想	bucket + chaining/tree	bucket + chaining
Dynamic resize	✓	✓
默认 load factor	0.75	0.75
默认初始 capacity	16	11
Synchronization	不同步	synchronized
null key/value	支持	不支持

同样是 O(1)，为什么实际性能不同？

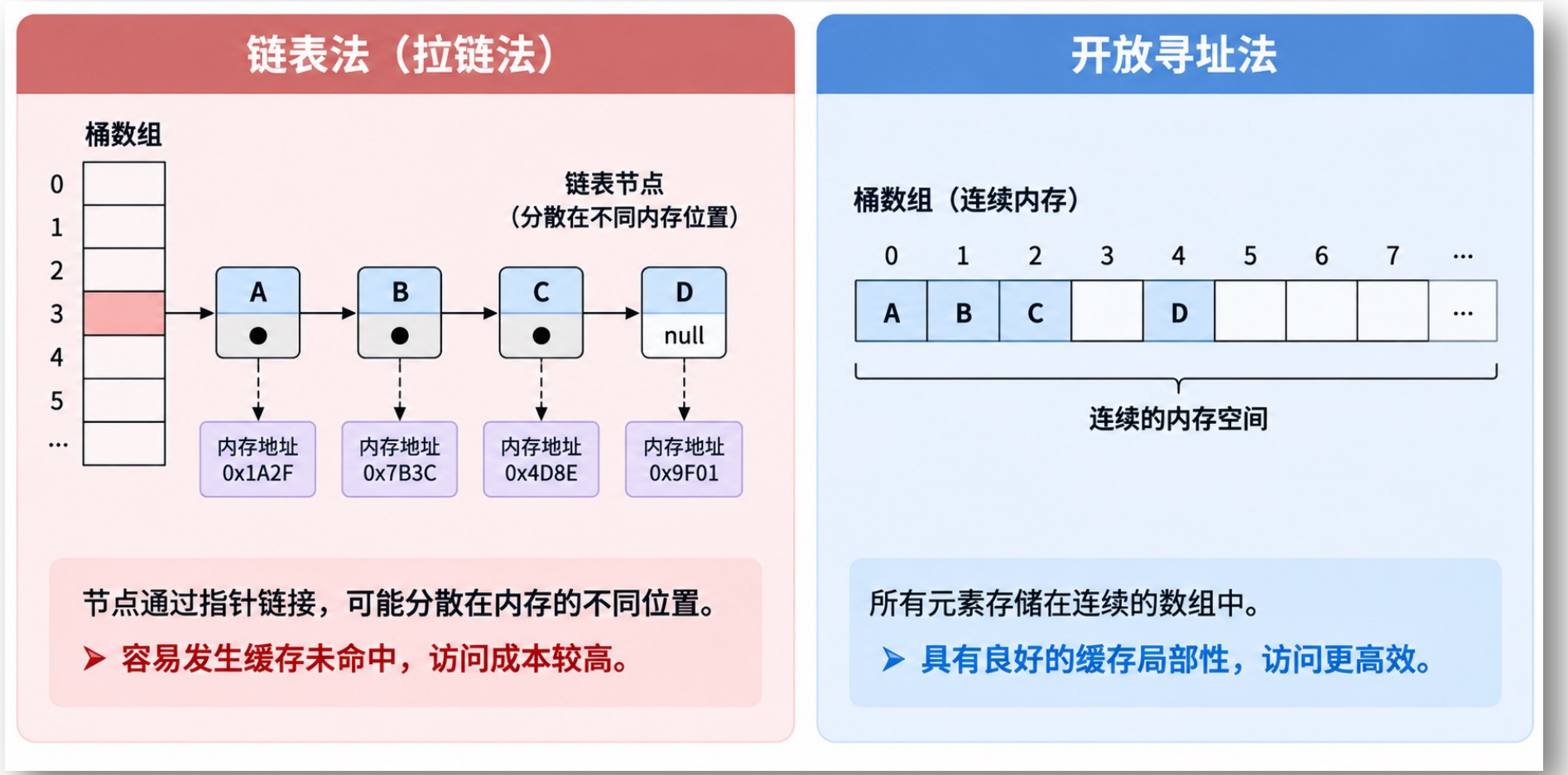
- Java HashMap 采用数组+链表/红黑树解决碰撞。查询复杂度已经是期望 $O(1)$，还能继续优化吗？

缓存命中

探查距离

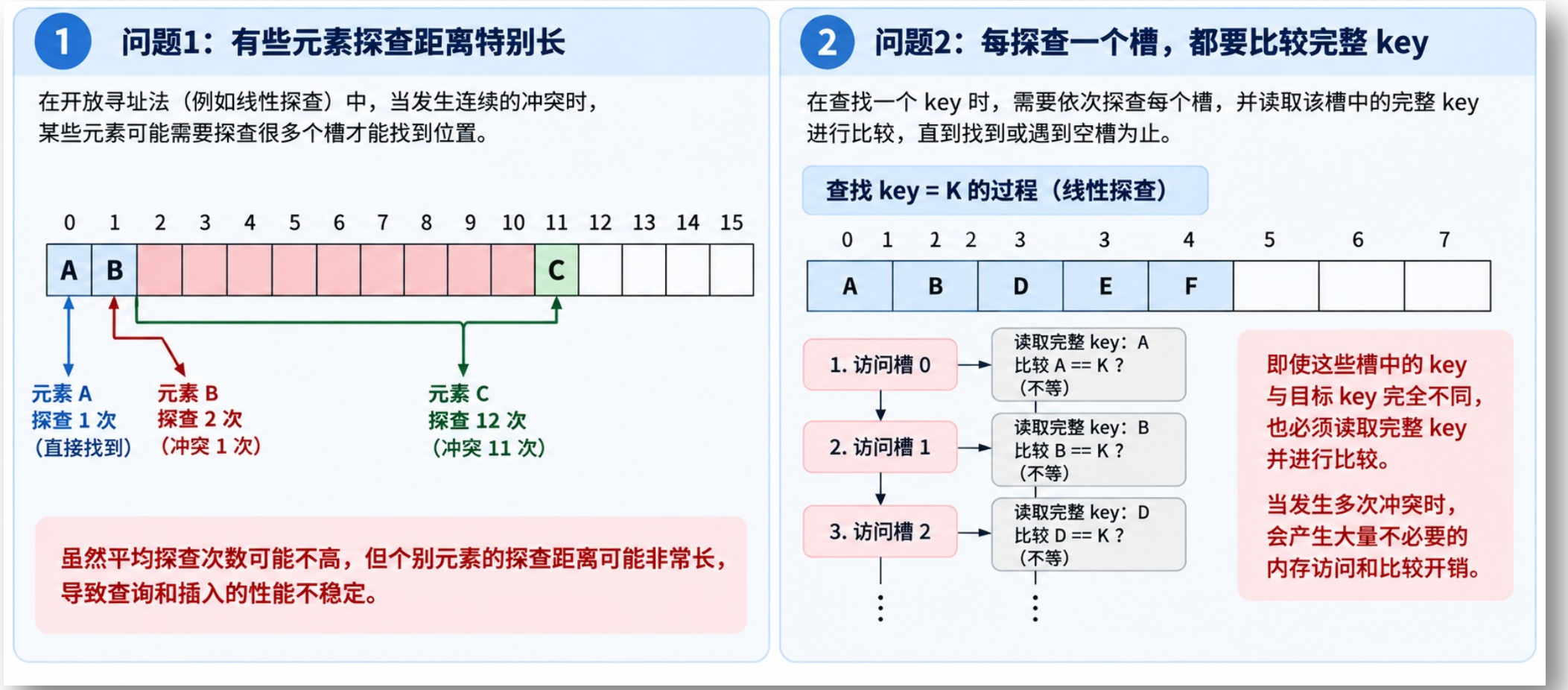
比较次数

内存瓶颈



开放寻址具有更好的缓存局部性

开发寻址法的问题



现代哈希表有两个主要优化方向：**1. 让探查距离更均匀，2. 减少完整Key的访问**

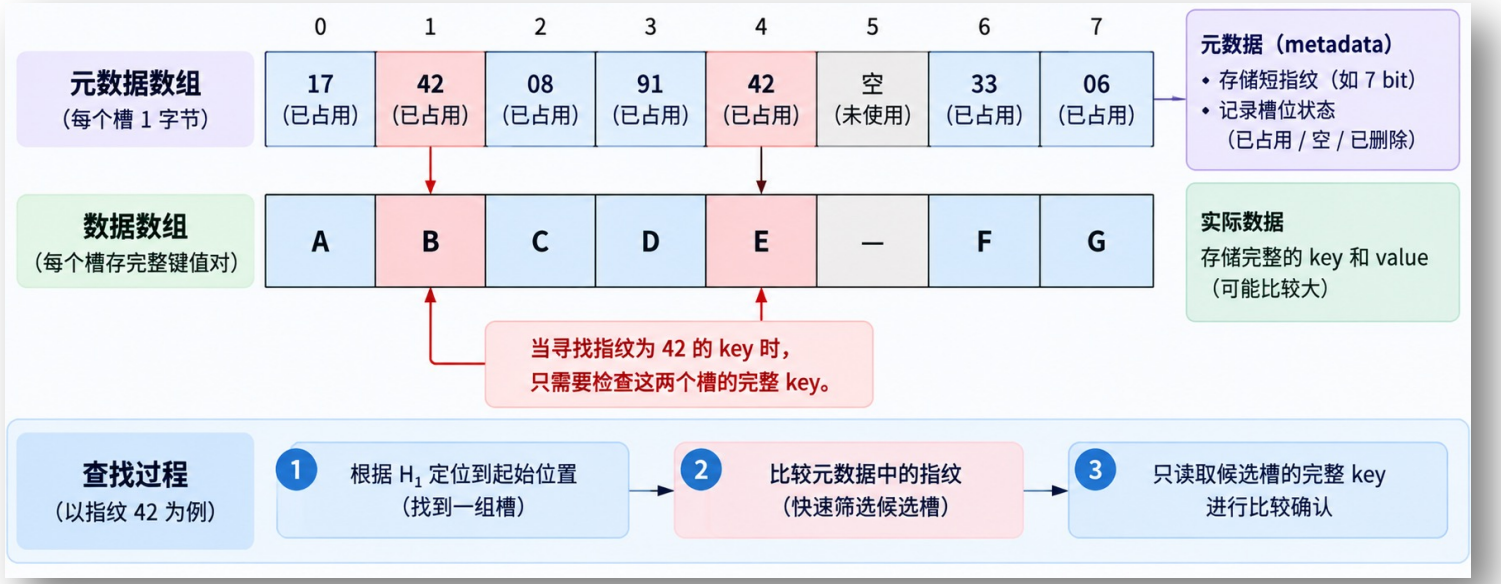
让探查距离更均匀

- 罗宾汉哈希：不要让某些 key 特别倒霉
- Robin Hood 的来源：**“劫富济贫”**
- 这里劫持的是：
  - 探查距离 = 当前位置 − 理想位置
  - **富人**：离 home 很近，很幸运；
  - **穷人**：已经探查很远，很倒霉
- 不是追求
  - 平均探查次数大幅下降
- 而是追求：
  - 探查距离更均匀，最大探查距离更小



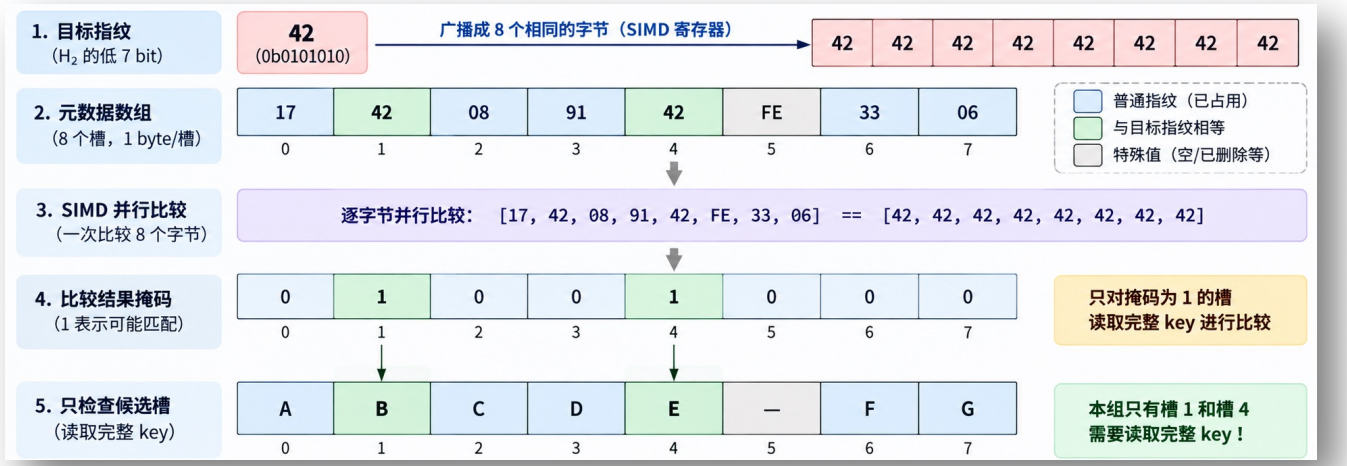
减少完整Key的访问

- 元数据/短指纹：把同一个 hash 值拆成两部分来用：$\boxed{\text{Hash} \rightarrow H_1 + H_2}$
  - H_1：决定**去哪里找**，相当于传统 hash table 的位置；
  - H_2：取 hash 的少量 bit，作为这个 key 的**短指纹**，存到槽位旁边的元数据里。
- 比如一个 key：$hash(k) = \underbrace{101101 \cdots 0010}_{H_1}\ \underbrace{0101010}_{H_2}$，$H_2 = 42$



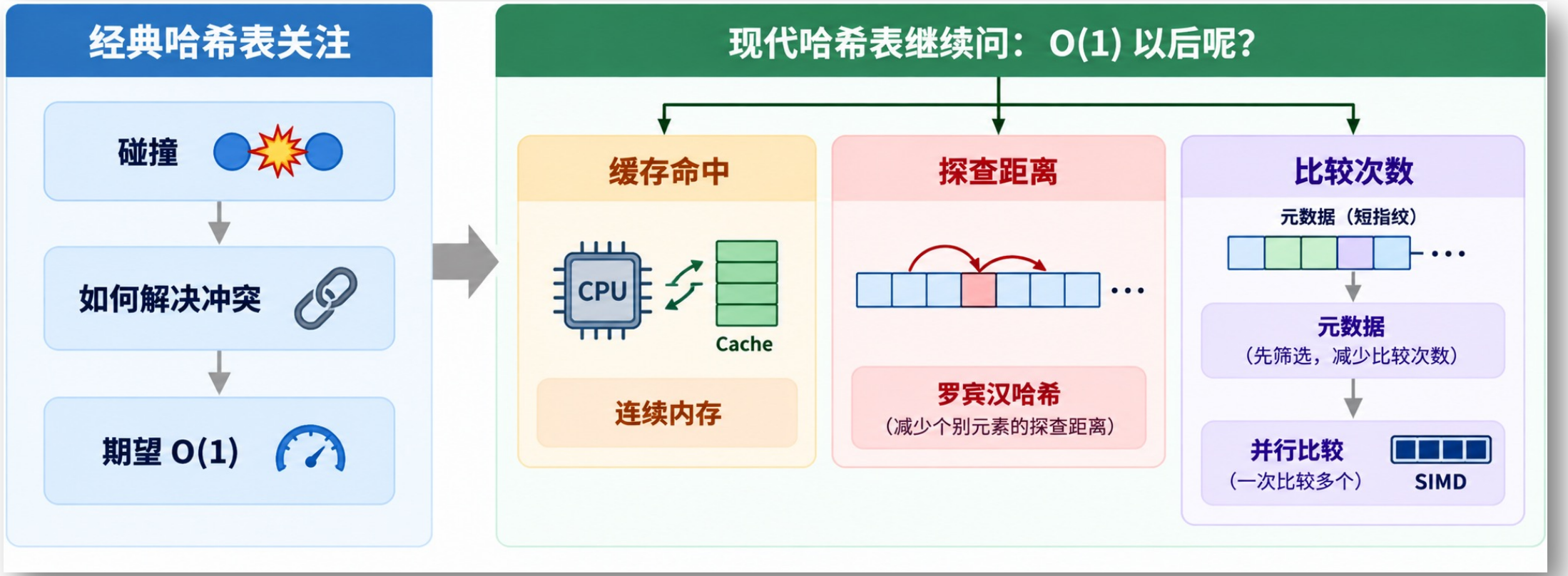
并行比较

操作	元数据	完整 key
数据大小	很小，通常约 1 Byte/槽	可能几十甚至几百 Byte
内存布局	连续	key/value 数据更大
CPU Cache	一次可装很多个	能装的数量少
比较方式	可一次并行比较多个	通常要逐个确认
字符串等复杂 key	不受 key 长度影响	可能逐字符比较



瑞士表 = 开放寻址 + 元数据/短指纹 + 分组探查 + 并行比较

现代哈希表


经典哈希表关注
碰撞
如何解决冲突
期望 O(1)
现代哈希表继续问：O(1) 以后呢？
缓存命中
CPU
Cache
连续内存
探查距离
罗宾汉哈希
（减少个别元素的探查距离）
比较次数
元数据（短指纹）
元数据
（先筛选，减少比较次数）
并行比较
（一次比较多个）
SIMD

确定性哈希函数的问题

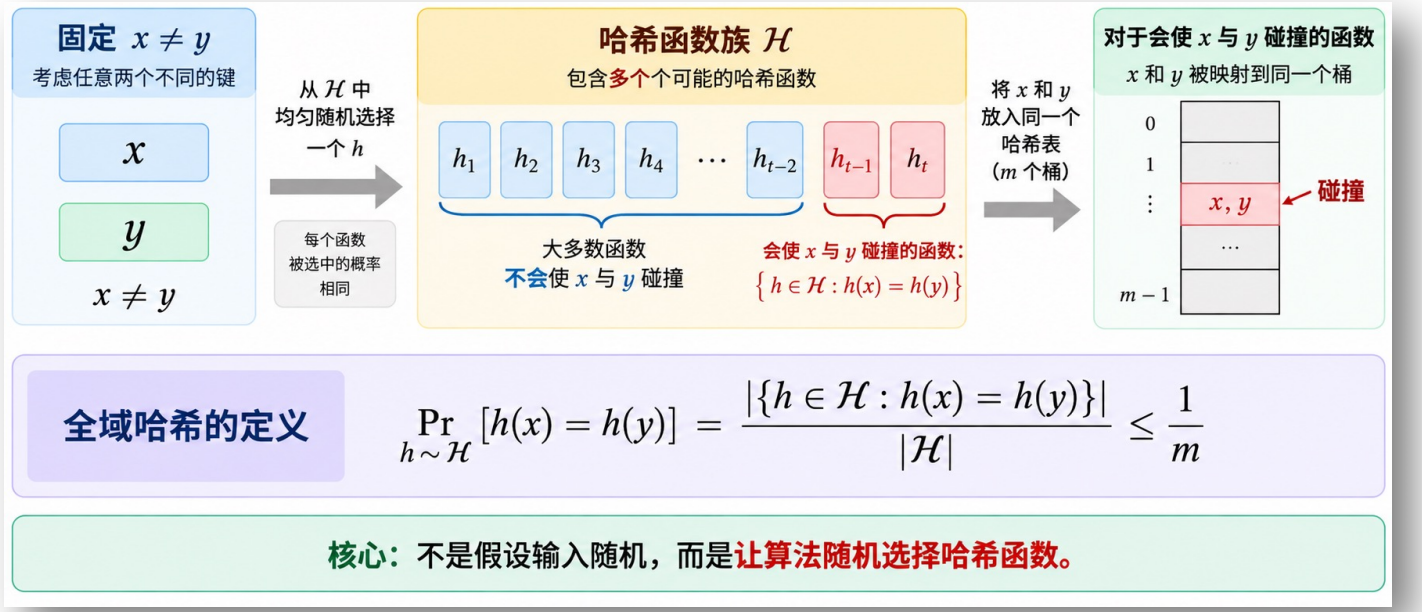
- **问题：**对于任何哈希函数h，存在一组键值可能导致哈希表的平均访问时间急剧上升
  - 如果哈希函数固定，聪明的对手可以故意挑选一些“坏键”：比如所有哈希值都等于某个槽位 i 的键。这样所有元素都会挤在槽位 i，查找/插入就会退化到 $O(n)$
- **思路：**随机选择哈希函数，独立于键值
  - 即使对手能看到哈希函数的代码，也无法找到一组不利的键值，因为不知道随机算法将会选择哪个哈希函数
- 所以：全域哈希（Universial Hashing）

全域哈希

- **全域哈希（Universal Hashing）**：设U为键的全集，H为一个有限的哈希函数族，其中每个 $h: U \to \{0,1,\dots,m-1\}$若对于任意不同的 $x, y \in U$，都有 $|\{h \in H: h(x) = h(y)\}| \leq \frac{|H|}{m}$，则称$H$是一个**全域哈希函数族（universal family of hash functions）**。

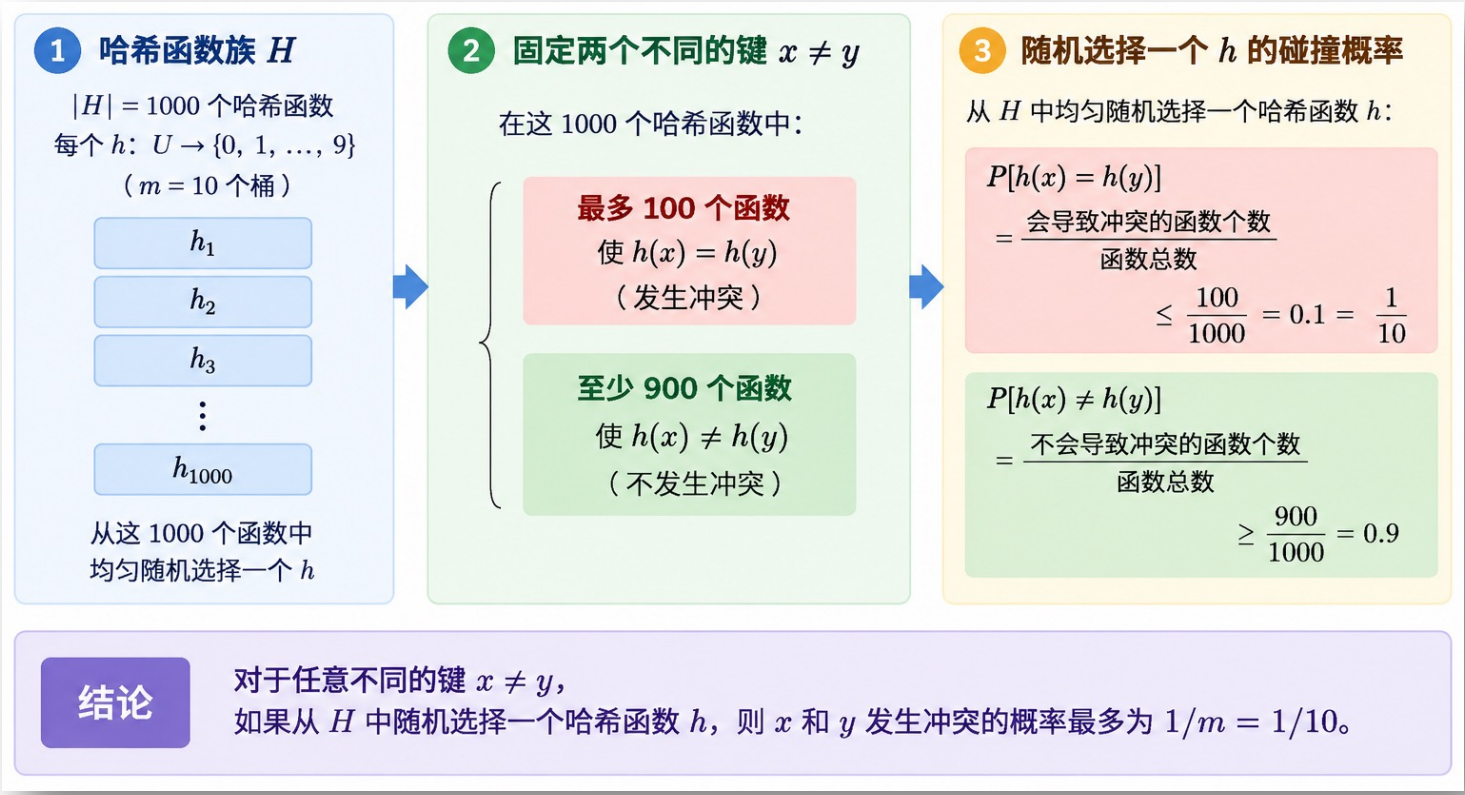
等价于一个更直观的写法：$\boxed{\Pr_{h\sim H}[h(x) = h(y)] \leq \frac{1}{m}}$

如果从H中随机选择一个哈希函数h，则x和y之间发生冲突的概率最多为$\frac{1}{m}$



例子

- 假设 $|H| = 1000$，$m = 10$。定义保证：对任意 $x \neq y$，最多有 $1000/10 = 100$ 个 h 让它们撞同一桶。
- 随机挑一个 h 的碰撞概率 $\leq 100/1000 = 0.1 = 1/10$



全域哈希的特性

- **定理：**设 H 是一个全域哈希函数族，h从H中均匀随机选取。将n个关键字映射到m个槽位。对于其中一个固定关键字x，则与x发生碰撞的其他关键字个数的期望满足 $\mathbb{E}[x \text{ 的碰撞次数}] \leq \frac{n-1}{m} < \frac{n}{m}$
- 假设现在有：$n = 100$ 个 key, $m = 10$ 个槽位，先固定一个 key：x，剩下还有 99 个 key：$y_1, y_2, \ldots, y_{99}$，对于每一个 y_i，因为 H是全域哈希族：$P\big(h(x) = h(y_i)\big) \leq \frac{1}{10}$，也就是说，每个 y_i跟 x 撞车的概率最多 10%。于是 99 个 key 中，平均跟x撞车的数量最多就是：$99 \times \frac{1}{10} = 9.9$，所以：

$$\boxed{E[\text{与 } x \text{ 碰撞的 key 数}] \leq 9.9}$$

证明

- 设C_x为一个随机变量，表示在哈希表 T 中与给定键x发生碰撞的键值的总数：

$$c_{xy} = \begin{cases} 1 & h(x) = h(y) \\ 0 & otherwise \end{cases}$$

- 有$E[c_{xy}] = 1/\mathrm{m}$, $C_x = \sum_{y \in T-\{x\}} c_{xy}$
- $C_x = \sum_{y \in T-\{x\}} c_{xy} \Rightarrow E[C_x] = E\left[\sum_{y \in T-\{x\}} c_{xy}\right] = \sum_{y \in T-\{x\}} E[c_{xy}] = \sum_{y \in T-\{x\}} \frac{1}{m} = \frac{n-1}{m} < \frac{n}{m}$

构造全域哈希前置知识：逆元

- **定理**：设m为素数。对于任何 $z \in Z_m$（即模 m 的整数集合）且 $z \neq 0$，存在一个唯一的 $z^{-1} \in Z_m$ 使得$z \cdot z^{-1} \equiv 1 \ mod \ m$
- 例如：$m = 7$

z	1	2	3	4	5	6
z^{-1}	1	4	5	2	3	6

全域哈希函数构造

- 设m为素数。将键 k 分解为 $r+1$位，每个位的值属于集合$\{0,1,\ldots,m-1\}$:

$$k=\langle k_0,k_1,\ldots,k_r\rangle, \text{其中 } 0\le k_i<m$$

$$\text{等同于}\boxed{k=k_0+k_1m+k_2m^2+\cdots+k_rm^r}$$

例如$m=5$，假设 $k=83$，则$83=3+1\times5+3\times5^2$，所以可以写成 $k=\langle 3,1,3\rangle_5$

- 随机策略
  - 选择$a=\langle a_0,a_1,\ldots,a_r\rangle$，其中每个$a_i$从集合 $\{0,1,\ldots,m-1\}$中随机选择

- 哈希函数：$h_a(k)=\sum_{i=0}^{r}a_ik_i \mod m$
- 因为$a=\langle a_0,a_1,\ldots,a_r\rangle$ 的选择有m^{r+1}个，所以哈希函数集合$H=\{h_a\}$的大小：$|H|=m^{r+1}$

全域哈希函数构造

- **哈希函数集合$H = \{h_a\}$是全域哈希**
- **证明：**假设 $x = \langle x_0, x_1, \dots, x_r \rangle, y = \langle y_0, y_1, \dots, y_r \rangle$ 是不同的键值。即它们在至少1位上不同。
- 假设我们不失一般性地考虑在位置 0 上不同。我们要确定在多少个哈希函数 $h_a \in H$ 中，键值x和 y 会发生碰撞？
- 碰撞条件：$h_a(x) = h_a(y)$
- 展开得到：$\sum_{i=0}^{r} a_i x_i \equiv \sum_{i=0}^{r} a_i y_i \mod m \Rightarrow \sum_{i=0}^{r} a_i (x_i - y_i) \equiv 0 \mod m \Rightarrow a_0(x_0 - y_0) + \sum_{i=1}^{r} a_i (x_i - y_i) \equiv 0 \mod m \Rightarrow a_0(x_0 - y_0) \equiv -\sum_{i=1}^{r} a_i (x_i - y_i) \bmod m$
- 考虑到$x_0 \neq y_0$，一个逆元$(x_0 - y_0)^{-1}$一定存在，使得$a_0 \equiv \left(-\sum_{i=1}^{r} a_i (x_i - y_i)\right) \cdot (x_0 - y_0)^{-1} \mod m$
- 因此，对于任何$a_1, a_2, \dots, a_r$的选择，恰好有一个对应的a_0的选择会导致x和y发生碰撞。（一旦$a_1, \dots, a_r$被确定，a_0就被唯一确定。）

全域哈希函数构造

- $a_1, \ldots, a_r$一共有r个系数，每个都有m种选择：m^r种。对于每一种 $(a_1, \ldots, a_r)$, 恰好有 **1 个** a_0会导致 x, y碰撞。因此导致碰撞的 a 一共有：$\boxed{m^r}$
- 而整个哈希函数族的大小为 $|H| = m^{r+1}$，所以：$|\{h_a \in H : h_a(x) = h_a(y)\}| = m^r$，而$\frac{|H|}{m} = \frac{m^{r+1}}{m} = m^r$

  于是：$\boxed{|\{h \in H : h(x) = h(y)\}| = \frac{|H|}{m}}$
- 满足全域哈希定义要求，得证

相关案例

- **真实案例：Hash Collision DoS**
- 2011 年研究人员公开展示了一类针对 Web 平台的 Hash Collision DoS：
- 攻击者构造大量**具有相同哈希值的 POST 参数名**，使服务器内部哈希表产生严重碰撞：

$$\text{正常：接近 } O(n) \quad \longrightarrow \quad \text{攻击：} O(n^2)$$

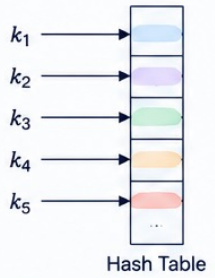
- 单个恶意 HTTP 请求就可能造成长时间 CPU 满载。
- Java、PHP、Python、Ruby 等多个平台受到影响。
- 后续防御包括：**Hash随机化、限制输入参数数量、碰撞链树化等。**
- **核心思想与全域哈希一致：** 让攻击者无法预知实际使用的哈希映射

工程实践

1 固定哈希函数：可能被攻击
Fixed Fixed Function: Vulnerable to Attack

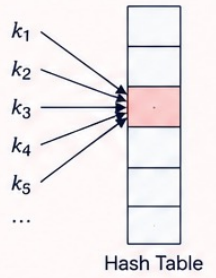
如果哈希函数是固定且公开的，
攻击者可以预先构造大量碰撞输入，
使哈希表从期望 $O(1)$ 退化到 $O(n)$。

正常情况（随机输入）



元素分布较均匀
期望操作时间：$O(1)$

攻击场景（构造输入）



大量输入落在同一个桶
操作时间退化到：$O(n)$



Hash Flooding / Hash DoS
攻击者利用哈希碰撞造成性能下降
甚至拒绝服务。

2 理论基础：全域哈希
Universal Hashing (Theory)

设计一个函数族 $\mathcal{H}$，运行时随机选择一个 hash function，使得对任意 $x \neq y$：

$$\Pr_{h \in \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{m}$$

典型构造（例）：

$$h_{a,b}(k) = ((ak + b) \bmod p) \bmod m$$

其中 a, b 随机选择。

核心思想

- 不依赖某一个固定的 hash function；
- 通过随机化，让攻击者难以构造大量碰撞；
- 提供概率上的碰撞保证。

适用场景

- 随机化算法分析
- Streaming / Sketch（如 Count-Min Sketch）
- 密码学（如 Wegman–Carter MAC）

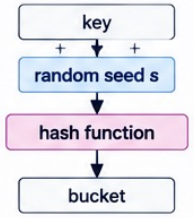
3 工程实践：随机种子 / 密钥哈希
Random Seed / Keyed Hashing (Practice)

在实际系统中，通常使用固定的哈希算法，
但为每次运行选择一个随机 seed，或使用一个秘密 key。

$h_s(k)$ 或 $h_K(k)$

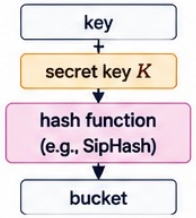
算法可以公开，
但随机参数（seed / key）
对攻击者保密。

随机种子（Random Seed）
程序自动时生成随机 seed s



同一个 key 在不同运行中
可能得到不同的 hash 值。

密钥哈希（Keyed Hashing）
使用秘密 key K（需要保密）



即使攻击者知道算法，
不知道 key 也难以构造碰撞。

例子：SipHash

- 一种快速的 keyed hash function
- 专为哈希表在对抗性输入下的使用而设计
- 被多种语言和系统（如 Python、Ruby 等）采用

整体思路：从理论到实践

固定哈希函数（可能被攻击） → 全域哈希（理论基础） → 随机种子 / 密钥哈希（工程实践） → 抵抗预构造碰撞（防止 Hash DoS）

核心结论

不要让输入能够针对一个固定的 hash function。

Don't let the input target a fixed hash function.

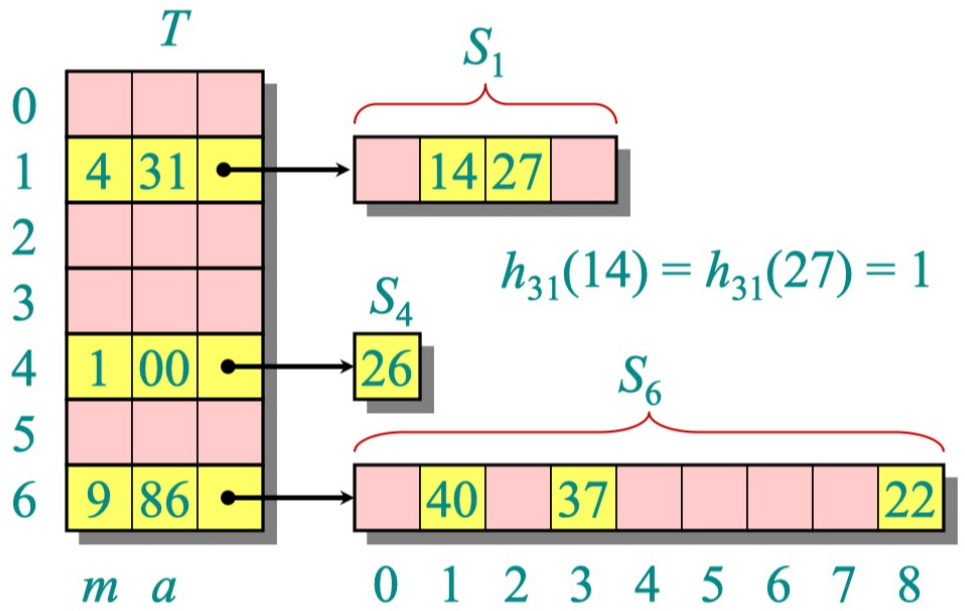
完全哈希

- 既然我们已经可以随机选择哈希函数，那能不能更进一步——直接找到一个对当前这批 key 完全不冲突的哈希函数？

- **完全哈希（Perfect Hashing）：** 对于一个**静态的**、包含 n个键的集合，构造哈希表，使得对任意键的查找操作都能在 $\Theta(1)$的**最坏情况时间**内完成。

完全哈希

- **思路：**思路：采用两层全域哈希。第一层允许碰撞，把键分到多个桶；第二层为每个桶单独构造一个无碰撞哈希表，从而实现最坏情况 $O(1)$ 查找。



完全哈希

- **定理**：设第i个第一层 bucket 中有 n_i个键。为该 bucket 建立一个大小为 $m_i = n_i^2$的第二层哈希表，并从一个全域哈希函数族 H_i 中均匀随机选择哈希函数 h_i。记 C_i 为该 bucket 中发生碰撞的**键对数量**，则

$$\mathbb{E}[C_i] \leq \binom{n_i}{2}\frac{1}{m_i} = \frac{n_i(n_i-1)}{2n_i^2} < \frac{1}{2}$$

- 根据全域哈希函数的定义，给定的两个键 x和y在哈希表中发生碰撞的概率是$\frac{1}{m}$，其中 $m = n^2$ 。因此，发生碰撞的概率是$\frac{1}{n^2}$。考虑所有可能的键对，共有$\binom{n}{2}$对键可以碰撞，则期望碰撞数目为

$$\binom{n}{2}\frac{1}{n^2} = \frac{n(n-1)}{2}\cdot\frac{1}{n^2} < \frac{1}{2}$$

完全哈希

- **推论：**第二层随机选择哈希函数时，没有碰撞的概率至少为$1/2$
- 证明：基于马尔可夫不等式：$\Pr\{X \geq t\} \leq \frac{E[X]}{t}$,X是非负的随机变量
- $E[X] = \sum_{x=0}^{\infty} x \cdot \Pr\{X = x\} \geq \sum_{x=t}^{\infty} x \cdot \Pr\{X = x\} \geq \sum_{x=t}^{\infty} t \cdot \Pr\{X = x\} = t \cdot \Pr\{X \geq t\} \Rightarrow \Pr\{X \geq t\} \leq \frac{E[X]}{t}$

- 应用这个不等式，有$\Pr\{X \geq 1\} \leq E[X] < \frac{1}{2}$
- 通过在H中测试随机哈希函数，很快会找到一个有效的哈希函数。

存储空间

- 对于第1层哈希表T，选择 $m = n$，并让n_j为哈希到T中第j个槽位的键的数量的随机变量。通过使用$m_j = n_j^2 (j = 0,1, \dots, \mathrm{m}-1)$个槽位来构建第二层哈希表$S_j$，所需的期望总存储量为$E\left[\sum_{j=0}^{m-1} m_j\right] = E\left[\sum_{j=0}^{m-1} n_j^2\right]$
- **上界：** $E\left[\sum_{j=0}^{m-1} n_i^2\right] < 2n$
- **证明：** $a \geq 0, a^2 = a + 2\binom{a}{2}$ 一定成立
- $E\left[\sum_{j=0}^{m-1} n_j^2\right] = E\left[\sum_{j=0}^{m-1} n_j + 2\binom{n_j}{2}\right] = E\left[\sum_{j=0}^{m-1} n_j\right] + 2E\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] = E[n] + 2E\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] = n + 2E\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right]$
- $\sum_{j=0}^{m-1}\binom{n_j}{2}$是哈希表中发生碰撞的关键字的总对数，根据全域哈希性质，我们有
- $E\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] \leq \binom{n}{2}\frac{1}{m} = \binom{n}{2}\frac{1}{n^2} = \frac{n-1}{2} \Longrightarrow E\left[\sum_{j=0}^{m-1} n_j^2\right] = n + 2E\left[\sum_{j=0}^{m-1}\binom{n_j}{2}\right] \leq n + n - 1 = 2n$
- 存储二次哈希表的空间期望值小于$2n = O(n)$

动态完全哈希

- **思考：**如何构造动态完全哈希？
- 答案：必要时“重建”

操作	复杂度
SEARCH(x)	最坏情况 O(1)
INSERT(x)	期望摊还 O(1)
DELETE(x)	期望摊还 O(1)
空间	O(n)

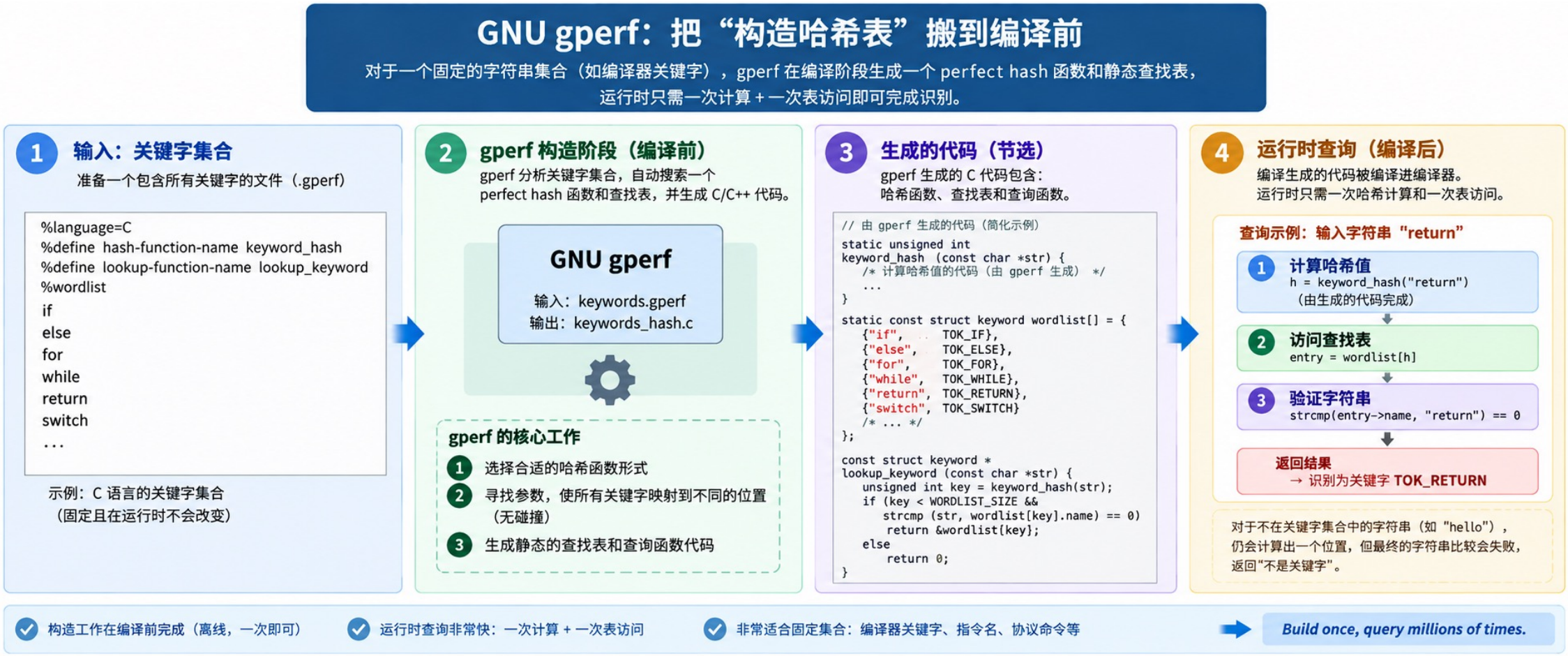
应用：编译器每天都在做的一件事

- 词法分析器每读到一个字符串，都需要判断：
- **它是不是语言的保留关键字？**
  - if → KEYWORD_IF
  - while → KEYWORD_WHILE
  - return → KEYWORD_RETURN
  - hello → IDENTIFIER
  - count → IDENTIFIER

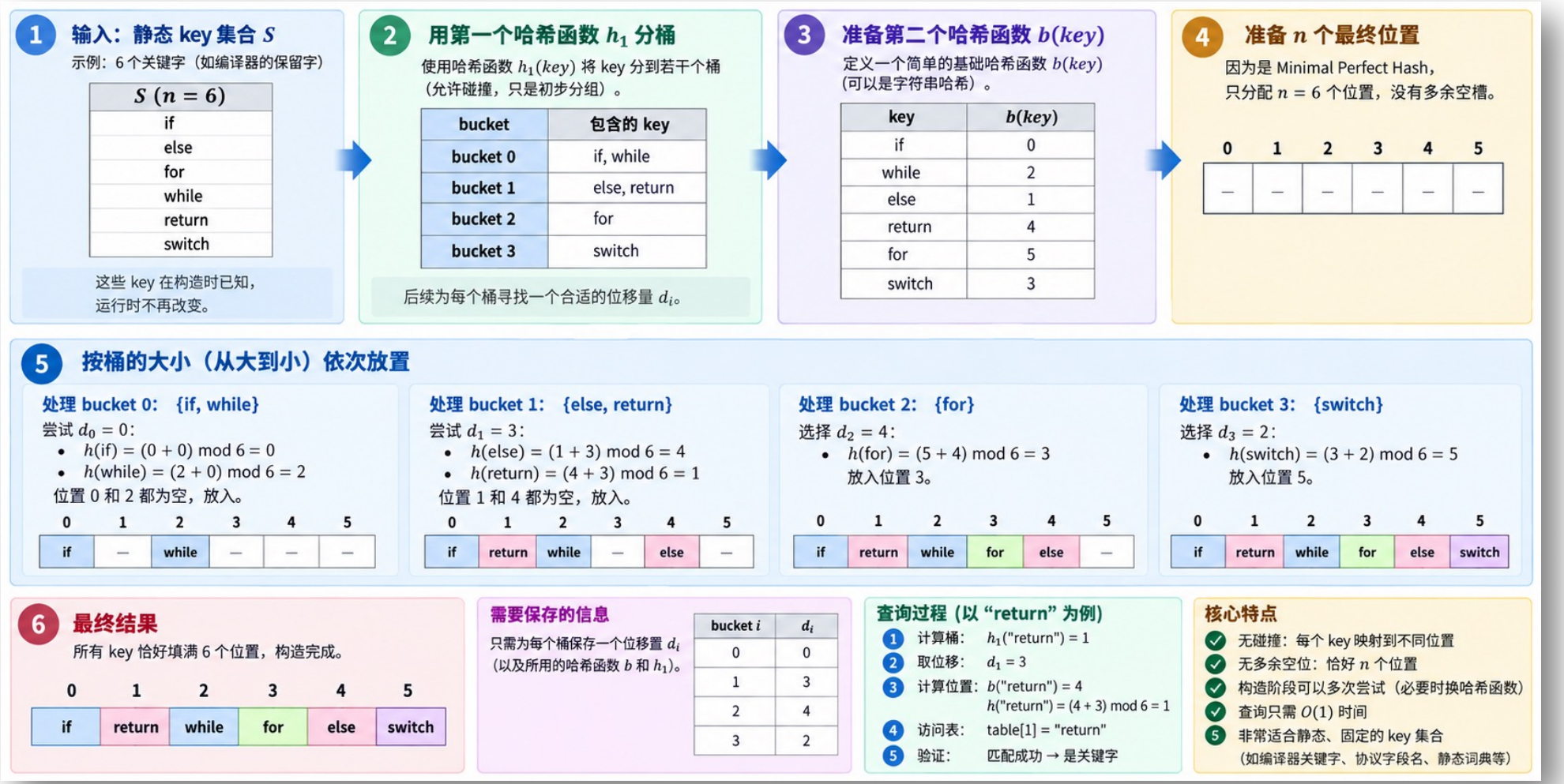
```
if (x > 0) {
        while (y > 0)
                return y;
}
```

GNU gperf

- gperf 是开发编译器时用的代码生成工具：根据固定关键字集合，提前生成高效的关键字查找代码，并编译进最终的编译器。



最小完全哈希


1 输入：静态 key 集合 S
示例：6 个关键字（如编译器的保留字）
$S\ (n=6)$
if
else
for
while
return
switch
这些 key 在构造时已知，
运行时不再改变。
2 用第一个哈希函数 h_1 分桶
使用哈希函数 $h_1(key)$ 将 key 分到若干个桶
（允许碰撞，只是初步分组）。
bucket
包含的 key
bucket 0
if, while
bucket 1
else, return
bucket 2
for
bucket 3
switch
后续为每个桶寻找一个合适的位移量 d_i。
3 准备第二个哈希函数 $b(key)$
定义一个简单的基础哈希函数 $b(key)$
（可以是字符串哈希）。
key
$b(key)$
if
0
while
2
else
1
return
4
for
5
switch
3
4 准备 n 个最终位置
因为是 Minimal Perfect Hash，
只分配 $n=6$ 个位置，没有多余空槽。
0 1 2 3 4 5
5 按桶的大小（从大到小）依次放置
处理 bucket 0： {if, while}
尝试 $d_0=0$：
h(if) = (0 + 0) mod 6 = 0
h(while) = (2 + 0) mod 6 = 2
位置 0 和 2 都为空，放入。
0 1 2 3 4 5
if – while – – –
处理 bucket 1： {else, return}
尝试 $d_1=3$：
h(else) = (1 + 3) mod 6 = 4
h(return) = (4 + 3) mod 6 = 1
位置 1 和 4 都为空，放入。
0 1 2 3 4 5
if return while – else –
处理 bucket 2： {for}
选择 $d_2=4$：
h(for) = (5 + 4) mod 6 = 3
放入位置 3。
0 1 2 3 4 5
if return while for else –
处理 bucket 3： {switch}
选择 $d_3=2$：
h(switch) = (3 + 2) mod 6 = 5
放入位置 5。
0 1 2 3 4 5
if return while for else switch
6 最终结果
所有 key 恰好填满 6 个位置，构造完成。
0 1 2 3 4 5
if return while for else switch
需要保存的信息
只需为每个桶保存一个位移量 d_i
（以及所用的哈希函数 b 和 h_1）。
bucket i
d_i
0
0
1
3
2
4
3
2
查询过程（以 “return” 为例）
1 计算桶： h_1("return") = 1
2 取位移： $d_1=3$
3 计算位置： b("return") = 4
h("return") = (4 + 3) mod 6 = 1
4 访问表： table[1] = "return"
5 验证： 匹配成功 → 是关键字
核心特点
无碰撞：每个 key 映射到不同位置
无多余空位：恰好 n 个位置
构造阶段可以多次尝试（必要时换哈希函数）
查询只需 $O(1)$ 时间
非常适合静态、固定的 key 集合
（如编译器关键字、协议字段名、静态词典等）

布谷鸟哈希

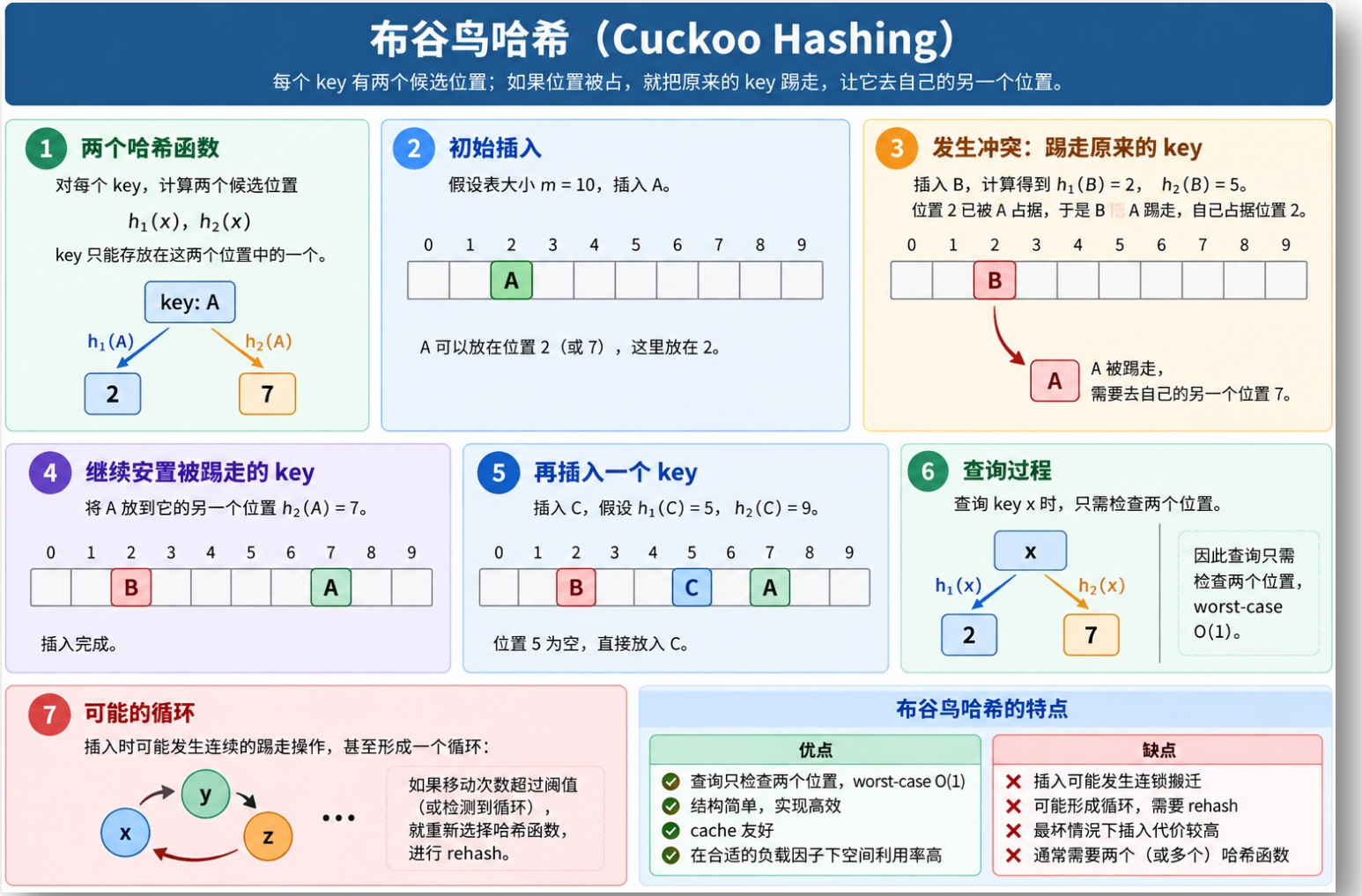
- 思考：既然碰撞麻烦，那能不能让每个 key 不只拥有一个位置，而是拥有两个候选位置；如果位置被占，就把原住民踢走？



布谷鸟有一个著名行为：
把蛋下到别的鸟巢里，甚至把原来的蛋挤出去。

布谷鸟哈希

- 为每个 key 准备两个哈希函数：$h_1(x), h_2(x)$
- 所以每个 key 只能住在两个位置之一：
- $\boxed{x \in T_1[h_1(x)] \quad 或 \quad x \in T_2[h_2(x)]}$



开销分析

	Search	Insert	Delete	额外代价
Cuckoo Hashing	$O(1)$，≤ 2 probes	Expected $O(1)$	$O(1)$	2 hash functions、relocation、偶发 rehash
最大优势	Lookup 极快且有确定的探测次数			
主要代价		插入可能触发连锁搬迁		表不能太满

练习题

- 设 $n = r^2$，其中 $r \geq 5$ 为整数。将关键字全集 $U = \{1, \ldots, n\}$ 中的全部键，存入一个有 n 个桶的链表哈希表，桶编号为 $0, \ldots, n-1$。每轮从同一个有限非空函数族 $\mathcal{H}$ 中独立、均匀地选取 $h: U \to \{0, \ldots, n-1\}$，重新建立整张表。函数族满足全域条件：$\forall x, y \in U,\ x \neq y,\ \Pr_{h \sim \mathcal{H}}[h(x) = h(y)] \leq \frac{1}{n}$。记 $L_h(x)$ 为键 x 所在链的长度（包含 x 自身），$M(h)$ 为整张表的最长链长度，即 $M(h) = \max_{x \in U} L_h(x)$。所有概率与期望均针对随机选取的 h。
- 某同学提出如下建表方案与分析：若 $M(h) > 4$，则重新选择哈希函数并重建，直到 $M(h) \leq 4$。由于每个事先固定的键 x 都有 $\mathbb{E}_h[L_h(x)] < 2$，所以每轮至少有 $1/2$ 的概率被接受，期望建表轮数不超过两轮。
- **（1）判断推断：**上述推断是否正确？给出理由，指出其逻辑问题。
- **（2）构造反例：**构造一个满足上述全域条件的函数族，使每个 $h \in \mathcal{H}$ 都有 $M(h) = r = \sqrt{n}$。描述函数族及其均匀抽样方式，证明全域性，并说明原建表方案能否终止。无须给出高效实现。
- **（3）修复方案：**将接受条件改为 $M(h) \leq 2\sqrt{n}$。证明：对任意满足题设条件的函数族，每轮接受概率至少为 $1/2$，期望建表轮数不超过两轮。

参考答案

- **（1）固定键的期望界，不是最长链的概率界**
- 推断不正确。对任意事先固定的 $x \in U$，由全域性与期望的线性性，确实有：$\mathbb{E}_h[L_h(x)] = 1 + \sum_{y \in U, y \neq x} \Pr_h[h(y) = h(x)] \leq 1 + \frac{n-1}{n} < 2$。马尔可夫不等式只能据此控制单个固定键：$\Pr_h[L_h(x) > 4] < 1/2$。接受条件却要求所有键所在的链同时不超过 4，即 $M(h) \leq 4$；前者不能推出后者。每个键都很少落入长链，并不意味着每次建表都没有长链。不同轮次落入长链的，可以是不同的键。
- **（2）反例：每轮随机选一批键挤入同一个桶**
- 按以下三步生成 h，令 $\mathcal{H}$ 为所有可能生成的映射组成的集合：
  - ① 从 U 的所有 r 元子集中均匀选取 S。
  - ② 从 n 个桶中均匀选取桶 t，令所有 $x \in S$ 满足 $h(x) = t$。
  - ③ 从将其余 $n - r$ 个键映射到桶 t 以外的 $n - 1$ 个桶的所有单射中，均匀选取一个，使这些键落入互不相同的桶。
- **均匀性。** 因为 $r > 1$，每个所得映射都有唯一的非单元素非空桶，其位置唯一确定 t，该桶中的键唯一确定 S；其余键的映射也唯一确定。每一步可选方案数只依赖 n, r，因此每个 $h \in \mathcal{H}$ 的生成概率相同。
- **最长链。** 每张表恰有一个长度为 r 的链，其余非空桶各有一个键，故对每个 $h \in \mathcal{H}$，都有 $M(h) = r = \sqrt{n}$。
- **全域性。** 对任意固定的 $x \neq y$，仅当二者同时属于 S 时才碰撞。利用 $n = r^2$，得：$\Pr_h[h(x) = h(y)] = \frac{r}{n} \cdot \frac{r-1}{n-1} = \frac{1}{r(r+1)} < \frac{1}{r^2} = \frac{1}{n}$.
- 所以该函数族满足全域条件。但 $r \geq 5$，没有任何函数满足 $M(h) \leq 4$，原方案必然无法终止。同时，对任意固定键 x，其进入长链的概率为 r/n，因此：$\mathbb{E}_h[L_h(x)] = \frac{r}{n} \cdot r + \left(1 - \frac{r}{n}\right) \cdot 1 = 2 - \frac{1}{r} < 2$.
- 数值例：$n = 10\,000$ 时，每个固定键所在链的期望长度为 1.99，但每张表的最长链长度恒为 100。

参考答案

- **（3）用链长平方和控制最长链**
- 设第 j 个桶的链长为 b_j（为简洁起见，省略其对 h 的依赖），定义：$Z = \sum_{j=0}^{n-1} b_j^2$, $C = \sum_{j=0}^{n-1} \binom{b_j}{2}$.
- 其中 C 是发生碰撞的无序关键字对数。每对不同键只计一次；由 $\sum_j b_j = n$，可得：$Z = \sum_{j=0}^{n-1} \left(b_j + 2\binom{b_j}{2} \right) = n + 2C.$
- 全域性保证每对不同键的碰撞概率至多为 $1/n$，故：$\mathbb{E}_h[C] = \sum_{1 \le x < y \le n} \Pr_h[h(x) = h(y)] \le \binom{n}{2} \frac{1}{n} = \frac{n-1}{2}$.
- 因此，链长平方和满足经典上界：$\mathbb{E}_h[Z] = n + 2\mathbb{E}_h[C] \le 2n - 1 < 2n.$
- 如果 $M(h) > 2\sqrt{n}$，则 $Z \ge M(h)^2 > 4n$。由事件包含关系和马尔可夫不等式：$\Pr_h[M(h) > 2\sqrt{n}] \le \Pr_h[Z > 4n] \le \frac{\mathbb{E}_h[Z]}{4n} < \frac{1}{2}$.
- 于是每轮接受概率 $p = \Pr_h[M(h) \le 2\sqrt{n}] > 1/2$。记 T 为直至首次接受所用的建表轮数（包括首次）。各轮独立且接受概率相同，故 T 服从几何分布：$\mathbb{E}[T] = \frac{1}{p} < 2.$
- 因此，题目要求的“期望建表轮数不超过两轮”成立。